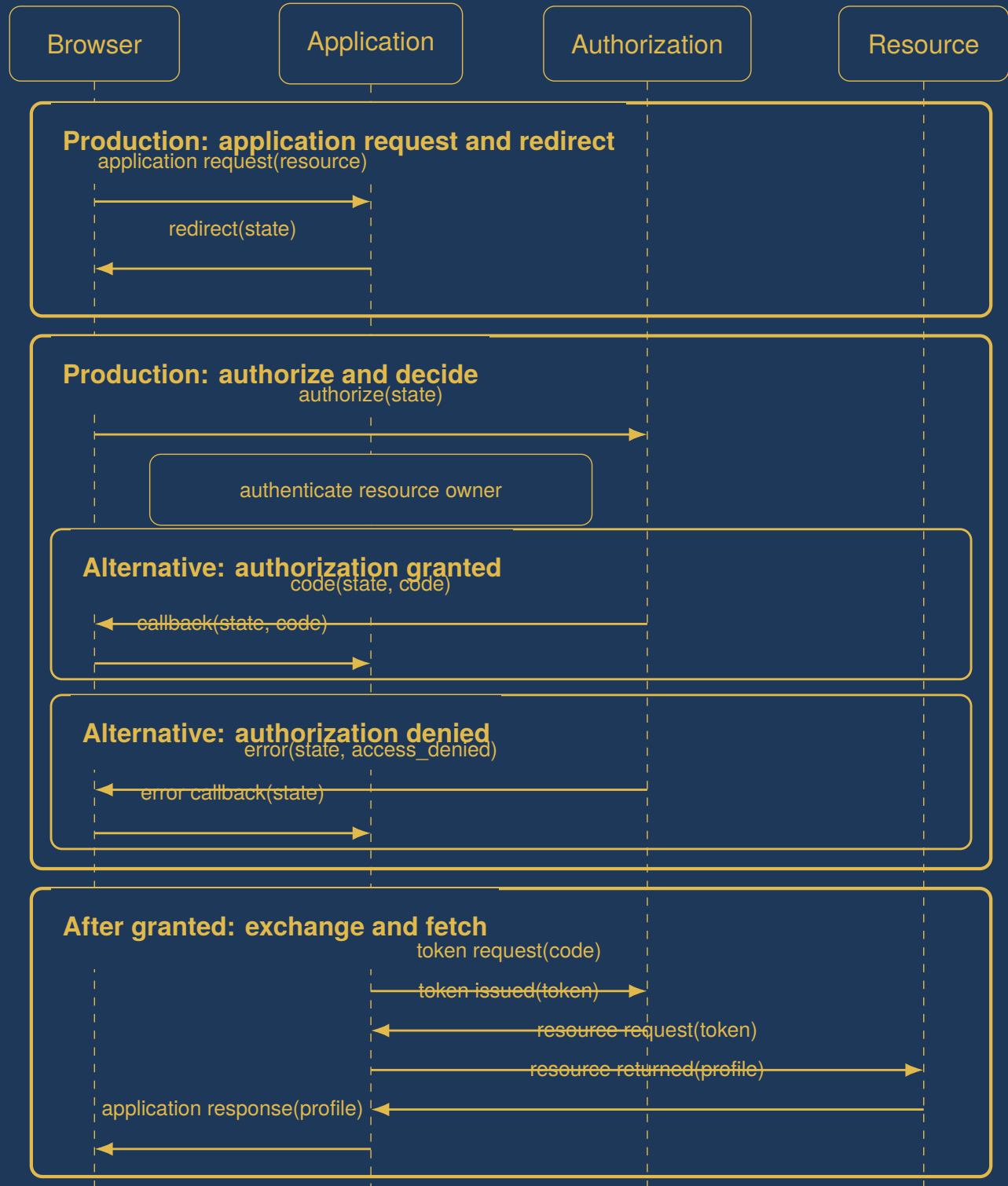


Formal Methods via LLM Grammars



Preface

Formal methods should be the easy way to build software. This book develops that claim into a practical method: describe externally meaningful behaviour as a parameterised, multiparty grammar; interpret its sentences as event traces; state temporal requirements in CTL; and render the whole specification as ordinary Lean. The specification says what participants may observe, not how an implementation stores every intermediate detail.

This is a working book and an executable design notebook. Definitions will migrate into a stable Lean library; systems built with the library should consist mostly of domain types, grammar productions, and theorems about expected behaviour. The evolving implementation, examples, and book sources are maintained in the LeanFM repository [2].

Contents

Preface	i
I The Grammar-Based Method	1
1 Formal Methods as the Default Path	2
1.1 Behaviour before machinery	2
1.2 Comparison with TLA+	2
1.3 Three ingredients	3
1.4 A test for every abstraction	4
2 Histories Made of Events	5
2.1 Compute everything from a fork/join event list	6
2.2 Derived state	8
2.3 Time is more than a timestamp	9
2.4 Round trip through bytes	9
3 The Bakery Event Corpus	10
3.1 The grammar represented by events	10
3.2 Money by day is the primary XY plot	11
3.3 Waste explains part of the margin	11
3.4 XY plots as questions over scenarios	12
3.5 AI rendering from durable requirements to pseudocode	12
4 A Multiparty Grammar of Behaviour	17
4.1 A small authoring algebra	17
4.2 A BNF for composable interactions	17
4.3 Parameters carry information	19
4.4 Composition without omniscience	19
4.5 Actors multiplex bounded conversations	20
5 The Scenario Authoring Language	22
5.1 The smallest useful scenario	22
5.1.1 Variables and assignment	22
5.2 Vague messages first, typed alphabets later	23
5.3 One concern per scenario	23
5.4 Interaction diagram: accepted quote	24

5.5	Choices and unresolved probability	25
5.6	Actor specifications and instances	25
5.7	Non-functional rates and scalability	26
5.7.1	The three USL regimes and capacity	26
5.8	Outages, recovery, and replica populations	29
5.9	Infrastructure cost and demand-constrained supply	29
5.10	Specified dwell time	30
5.11	Estimating completion from an event stream	30
5.12	Composing requirements means intersecting constraints	31
5.13	State-machine rendering	32
5.14	Charts are declarative trace reductions	32
6	Arguing Requirements through the Lean Web Server	35
6.1	The argument loop	35
6.2	The server is a staging environment	36
6.3	What the Lean artifact must settle	36
6.4	Interrogate, characterize, synthesize, replay	37
6.5	The protobuf artifact is part of the requirement	38
6.6	A ladder of checks	38
6.7	How to argue well	39
7	Worked Example: OAuth Authorization Code Flow	40
7.1	Actors, values, and observable events	40
7.2	Production 1: application request and redirect	41
7.3	Production 2: authorize, then grant or deny	42
7.4	Scenario 3: exchange the code and use the token	43
7.5	Reading the scenarios together	43
7.6	The parallel protobuf alphabet	44
7.7	Generate one legal trace and its artifacts	46
7.7.1	Envelope stream and concrete bytes	47
7.7.2	Residual state-machine path	47
7.7.3	Actor projections	48
7.7.4	Event order, queues, and memory	48
7.7.5	Measurements, plots, and verdicts	48
7.8	Interaction diagrams by scenario	49
7.9	Requirements as observable constraints	49
7.9.1	Who knows the password?	49
7.9.2	Byte-level refinement without choosing a transport	51
7.10	Probabilities and charts	51
7.11	Residual state machine	52
8	CTL over Generated Futures	54
8.1	Basic modal logic	54
8.2	LTL: time along one path	54
8.3	CTL: path modality combined with time	55
8.4	General and multiple modalities	56
8.5	Possibility means preparedness	56

8.6	Strong implication and the Chewbacca argument	57
8.7	Weak until: AW and EW	58
8.8	Past claims	58
8.9	Fairness and time bounds	58
9	Lean as the Rendered Language	59
9.1	From specification to service	59
10	Research and Implementation Roadmap	60
10.1	Milestones	60
10.2	Proof obligations	60
II	Security Protocol Exercises from the CSP Approach	62
11	From CSP Roles to Multiparty Grammars	63
11.1	Can actors really be specified separately?	63
11.1.1	Local state without global implementation state	63
11.2	A transport-neutral byte algebra	64
11.3	Yahalom as three separate actors	64
11.3.1	Alice, the initiator	64
11.3.2	Bob, the responder	65
11.3.3	The trusted server	65
11.3.4	Global assembly and context conditions	66
11.4	The attacker is another grammar plus deduction	66
11.5	Knowledge and agreement become theorems	67
11.6	Otway–Rees: guards are not optional decoration	67
11.7	Needham–Schroeder: public-key lookup as behaviour	68
11.8	What these exercises demand from the library	68
	Referenced Books	70
	Bibliography	72
	Index	73

Part I

The Grammar-Based Method

Chapter 1

Formal Methods as the Default Path

Formal methods are usually presented as an extra stage: first build a system, then translate a simplified version into a model, and finally ask a specialist tool a few questions. That workflow taxes correctness. The model drifts, the difficult translation is postponed, and proofs concern yesterday's architecture.

Our aim is the reverse. The shortest path to a running service should pass through an executable behavioural specification. Writing the specification should remove work: it supplies message types, scenario tests, model-checking input, documentation, and the skeleton of the service itself. Going without it should be the inconvenient choice.

1.1 Behaviour before machinery

The primary object is a set of permitted, visible histories. An implementation may use threads, actors, queues, databases, caches, or continuations. Those decisions matter operationally, but they are not automatically part of the contract. Promoting all internal state into the model creates a product state space whose size grows exponentially with independently varying components. Worse, it makes the specification brittle under harmless refactoring.

We therefore begin with *behaviour*: who communicated with whom, what kind of event occurred, what information it carried, and which predecessor order it extends. Internal state enters only when it changes an observable promise.

1.2 Comparison with TLA+

LeanFM is not an extension of TLA+ and does not try to replace it. TLA+ commonly describes a system with variables, an initial-state predicate, a next-state relation, and temporal formulas over the resulting state behaviors [6]. A message queue can be represented as a sequence-valued variable, and an action can append to or remove from that sequence. This is expressive and makes internal state transitions available to invariants and model checking.

LeanFM chooses a different primary boundary. The object available to us is often an event stream scraped from a running system. We can decode its messages, correlate them, follow predecessor backpointers, and classify the resulting sequence as in-spec or out-of-spec. We generally cannot observe the implementation's private variables at every step, so `Requirements.lean` does not pretend to maintain them. Its grammar both recognizes legal traces and generates them; state

machines are projections of visible histories rather than declarations of every variable stored by the program.

The distinction is methodological, not a claim that TLA+ is unable to express queues or traces. This project began with an attempt to extend a TLA+-style model with the desired message queues and MDP probabilities. The practical difficulty was upstream: explaining an ordinary real scenario required choosing and maintaining a small internal-state model before the observed behavior could even be recorded. The abstraction work could exceed the apparently trivial example. LeanFM instead asks first for the messages that would be visible in production.

The trade is deliberate:

Question	TLA+-style state specification	LeanFM event-language specification
Primary behavior	sequence of states	sequence or partial-order graph of events
Queue representation	usually an explicit variable	observable enqueue, send, receive, blocking, and capacity facts
Conformance input	modeled states or action correspondence	decoded production event stream
Hidden variables	available when declared	omitted unless they affect an observable contract
Generation	behaviors satisfying the state relation	words/traces generated by the grammar
Probabilities	require an additional probabilistic interpretation or tool	weights and observed distributions are first-class model data
Main risk	state model may drift from instrumentation	event model may omit a hidden cause needed for a property

This boundary also separates the two durable Lean files. `Requirements.lean` contains specified observable behavior: accepted messages, legal ordering, probabilities, timing and reliability constraints, and properties measurable from events. `Implementation.lean` contains the additional decisions needed to generate software: target language, files, transports, runtime APIs, and bindings. Existing source code may be supplied when revising an implementation, but it does not redefine the requirement.

The separation is designed for loss of conversational context. The original LLM conversation is expected to disappear repeatedly. After each argument, the current understanding must survive in the requirements, protobuf, and implementation files, with requirement-to-code justifications. A new model session reads those files and continues from them; it need not reconstruct intent from a transcript.

1.3 Three ingredients

The method combines three traditions.

1. Language theory gives compact generative descriptions of legal histories. Regular and context-free grammars provide the approachable starting point. Dana Richards and Henry Hamburger,

the author's college professors, are cited in Richards and Hamburger [9] as the source from which the author learned these methods, not as their originators. For the underlying habits of symbolization, validity, proof, quantification, and modal reasoning, we also draw on the accessible progression of Gensler [3].

2. Protocol analysis contributes explicit participants, adversarial choices, trace semantics, and tool-supported checking. The CSP treatment of Ryan et al. [10], including Steve Schneider, is cited as the source from which the author learned this treatment.
3. Temporal logic turns expectations into queries over the generated transition system. CTL is central because it distinguishes claims about some possible future from claims about every possible future [1].

1.4 A test for every abstraction

Each abstraction in this book should answer a practical question:

Can a developer state a meaningful behaviour once, compose it with other behaviours, check its consequences, and keep it stable while implementation details change?

If an abstraction does not improve that workflow, it does not belong in the core library.

Chapter 2

Histories Made of Events

Every execution is represented by an event sequence. A deliberately small common envelope makes histories comparable across protocols:

```
structure Event (Actor Session Task Kind Payload : Type) where
  timeAt  : ClockTimestamp
  id      : String
  priorIds : List String
  session : Session
  task    : Task
  from    : Actor
  to      : Actor
  kind    : Kind
  payload : Payload

abbrev Trace (Actor Session Task Kind Payload : Type) :=
  List (Event Actor Session Task Kind Payload)
```

The fields have different jobs. `id` identifies a visible fact; `priorIds` records zero or more immediate predecessor backpointers; `from` and `to` name the parties; `(session, task)` correlates independently progressing conversations; and `kind` supplies a stable discriminator. The payload is protocol-specific and carries the parameters that flow from one production to the next. `timeAt` is sampled from the model's shared monotonic clock. Its differences measure elapsed time; it is not a civil-time date and its origin is immaterial.

The predecessor links form a directed acyclic graph rather than a singly linked list. An initial event has no prior identifiers. A fork is represented by several later events that each name the same prior event. A join is represented by one event whose `priorIds` names every branch on which it depends. An end event of any kind may also name its corresponding start event. That backpointer makes elapsed time simply `end.timeAt - start.timeAt`, even when several attempts with the same `(session, task)` overlap. A backpointer to an already-ancestral start is a redundant ordering edge, but it is useful correlation data and does not alter the partial order. The edge says “ordered after,” not “caused by.” A causal interpretation requires a separate grammar or domain assertion.

For an m -of- n join, the event additionally records m , n , and the exact m branch-terminal IDs that satisfied the threshold. The selected IDs must be distinct members of `priorIds`. Merely seeing two predecessors is not enough to distinguish a two-of-three threshold from an ordinary two-branch all-of join.

2.1 Compute everything from a fork/join event list

Each event is one observation at one instant. Starting and completing are different messages, so an event does not contain an interval. Durations are derived by selecting the two boundary events and subtracting their timestamps.

```
structure AnalysisEvent where
  id      : EventId
  priorIds : List EventId
  timeAt  : ClockTimestamp
  bytes   : ByteArray
  reveals : List ValueRef

def elapsed (started completed : AnalysisEvent) : Duration :=
  completed.timeAt - started.timeAt
```

Consider the first order in `examples/bakery-events.jsonl`. Payment and baking fork after the order is placed and join before the order is accepted:

Event	Prior backpointers	timeAt	Meaning	Bytes	Reveals
e_0	<code>[]</code>	258726	order placed	85	order
e_1	<code>[e_0]</code>	258921	payment requested	71	price
e_2	<code>[e_0]</code>	259194	bake requested	73	product
e_3	<code>[e_1]</code>	262219	payment authorized	49	authorization
e_4	<code>[e_2]</code>	2055947	bake completed	55	completion
e_5	<code>[e_3, e_4]</code>	2056288	order accepted	57	outcome
e_6	<code>[e_5]</code>	2056291	delivery requested	54	delivery
e_7	<code>[e_6]</code>	2283253	courier assigned	40	courier
e_8	<code>[e_7]</code>	6946819	order delivered	65	outcome

The list order is one serialization convenient for storage. Event order is the transitive closure of `priorIds`. Both e_1 and e_2 cite e_0 , so they fork. Event e_5 cites both e_3 and e_4 , so it joins the branches. The following quantities are folds or graph queries over this one list:

$$\begin{aligned} \text{paymentTime}(e_1, e_3) &= e_3.\text{timeAt} - e_1.\text{timeAt} = 3298, \\ \text{latency}(e_0, e_8) &= e_8.\text{timeAt} - e_0.\text{timeAt} = 6688093, \\ \text{wireBytes} &= 85 + 71 + 73 + 49 + 55 + 57 + 54 + 40 + 65 = 549. \end{aligned}$$

Two events are concurrent in this order when neither is an ancestor of the other:

```
def concurrent (events : List AnalysisEvent) (a b : EventId) : Bool :=
  !reachable events a b && !reachable events b a
```

The maximum concurrency permitted by the trace is the width of this partial order: the size of its largest antichain. Here $\{e_1, e_2\}$ is an antichain, as is $\{e_3, e_4\}$, and no three events are pairwise concurrent. Thus the maximal order concurrency is two. Incomparable events may commute: swapping them in a linear serialization preserves the same partial order, provided each event still appears after all of its named predecessors. A timestamped point event has no active interval; runtime concurrency requires separate start and completion messages. This is stronger than counting adjacent list elements and different from measuring average runtime concurrency.

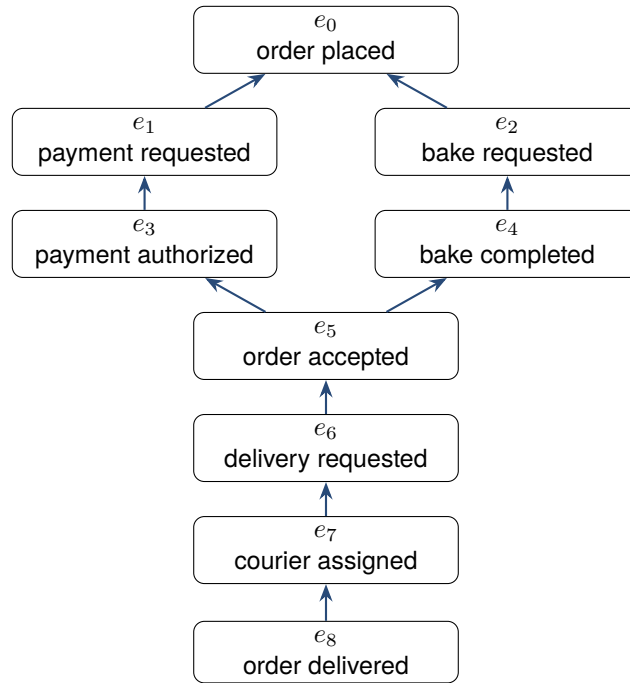


Figure 2.1: The table’s immediate predecessor backpointers rendered as an order graph. Each arrow points from an event to an ID in its `priorIds` list: for example, $e_2 \rightarrow e_0$. The two arrows from e_5 expose the payment/baking join.

Start and completion boundaries make nested concurrent work especially readable as an order graph. In the following trace, the request forks into two task attempts. Each completion points back to its own start, and the response joins the two completions. The shaded columns are visual grouping only; the arrows are still exactly the stored predecessor pointers.

Timestamps and predecessor links also give elapsed time along a selected path:

```

edgeTime(prior, event) := event.timeAt - prior.timeAt
elapsed(started, completed) := completed.timeAt - started.timeAt

```

The critical path is $e_0, e_2, e_4, e_5, e_6, e_7, e_8$, with elapsed time 6,688,093 ms. The payment branch finishes at tick 262,219 and has 1,793,728 ms of slack before the join. Queue occupancy, queued bytes, active (`session, task`) keys, and memory pressure are computed by adding enqueue/dequeue events to the same alphabet and folding them in clock order.

The entire checked-in corpus contains 20,000 orders and 171,991 events. Streaming that file produces the following model inputs; the numbers in this table are generated LaTeX macros, not independently typed constants:

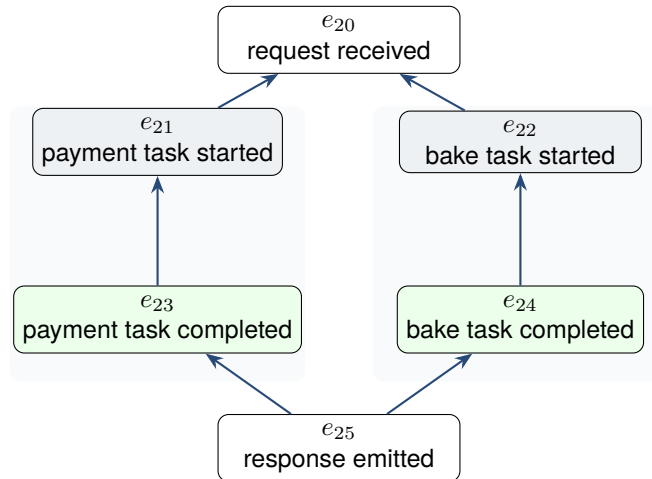


Figure 2.2: Task boundaries nested inside a concurrent fork/join. The intervals $e_{21} \dots e_{23}$ and $e_{22} \dots e_{24}$ may overlap, while the two branches remain incomparable in predecessor order. Either branch may appear first in a linear serialization without changing the graph.

Reduction over bakery-events.jsonl	Observed value
payment authorized / orders	18,788/20,000
bake completed / orders	19,393/20,000
order accepted / orders	18,217/20,000
delivered / orders	16,099/20,000
mean terminal latency	4,766,280.642 ms
median terminal latency	4,821,533 ms
95th-percentile terminal latency	7,863,807 ms
maximum simultaneously active orders	46

These ratios are suitable probability estimates for this corpus. Regeneration with a different source stream changes the macros and therefore changes every book example that consumes them.

Knowledge is another event reduction. For a value v , calculate the actors that can derive it from initial knowledge and the bytes visible to them:

```
def knowers (actors : List Actor) (v : ValueRef)
  (events : List AnalysisEvent) : List Actor :=
  actors.filter fun actor =>
    Derives (knowledgeFrom actor events) v
```

Encryption, signing, projection, key possession, and compromise contribute rules to `Derives`. The primary result is the set `knowers v events`. “Only these actors know v ” is a comparison between that computed set and an allowed set, not a separate primitive called secrecy.

2.2 Derived state

A current session, an authenticated principal, or an outstanding request can often be computed by folding the trace. Such a fact need not also exist as primitive model state. This discipline has

two benefits: evidence for a fact remains visible, and the state representation does not multiply independently of the history that justifies it.

Some implementations necessarily maintain private state. The model includes it only when two executions with the same visible trace may make different future behaviour legal. Even then, the first remedy is to expose the relevant distinction as an event or a grammar parameter rather than copying an implementation object graph into the specification.

2.3 Time is more than a timestamp

The central modelling problem is maintainable time evolution. We separate three ideas that are often conflated:

- sequence order: one visible event occurs before another;
- declared dependency or causality: a grammar says that one event extends or responds to another;
- duration: an amount of physical or logical time elapses.

Grammar structure declares legal sequence and domain dependency, `priorIds` encodes the resulting predecessor order, and typed constraints handle duration. A predecessor link alone is not evidence of causation. This keeps a change in timeout policy from rewriting the protocol's entire behavioural shape.

For two events from the same `(session, task)`, elapsed time is defined when the first event's `timeAt` value does not exceed the second's:

```
def elapsed (start finish : Event) : Option Duration :=
  if start.session = finish.session &&
    start.task = finish.task &&
    start.timeAt <= finish.timeAt
  then some (finish.timeAt - start.timeAt)
  else none
```

Latency queries name their boundary event kinds and follow the end event's backpointer to the start before subtracting timestamps. The `(session, task)` values must agree, but are not sufficient to choose a start when attempts overlap. Ordering comes from `priorIds`; any causal meaning comes from the grammar or another explicit domain relation. Equal timestamps may occur, and neither a timestamp nor a predecessor pointer alone establishes that one event caused another.

2.4 Round trip through bytes

A generated scenario is not complete until it can cross the byte boundary in both directions. `ScenarioEvent` in `Requirements.proto` carries the event identity, complete `prior` list, correlation key, actors, `timeAt`, and the selected protobuf atom. Encoding a `Scenario` therefore produces bytes for both its ordering envelope and payload. Decoding those bytes must reconstruct the same ordered event list. The executable round-trip check compares the decoded protobuf value with the generated value and separately checks the reconstructed `prior` graph; loss of a fork or join fails the build.

Chapter 3

The Bakery Event Corpus

The bakery is the quantitative worked example. Its source is not a table of percentages copied into this chapter. It is the 171,991-line `examples/bakery-events.jsonl` stream. The reducer reads those events and generates both `examples/bakery-stats.json` and the coordinates plotted below. Changing the stream changes the book.

3.1 The grammar represented by events

An order begins with `OrderPlaced`. Payment and baking fork from that event, and `OrderAccepted` or `OrderRejected` joins their result events. An accepted order either resolves to customer cancellation or continues through courier assignment to delivery or delivery failure. The file also contains `ShiftPaid` events, one per employee shift. Thus operational and accounting facts share the same event algebra without being the same task.

For each calendar-day bucket d , the reducer computes

$$\begin{aligned} \text{grossSales}_d &= \sum_{e.\text{kind}=\text{PaymentAuthorized}} e.\text{amount}, \\ \text{refunds}_d &= \sum_e e.\text{refund}, \\ \text{netRevenue}_d &= \text{grossSales}_d - \text{refunds}_d, \\ \text{ingredientCost}_d &= \sum_{e.\text{kind} \in \{\text{BakeCompleted}, \text{BakeFailed}\}} e.\text{ingredientCost}, \\ \text{employeePay}_d &= \sum_{e.\text{kind}=\text{ShiftPaid}} e.\text{pay}, \\ \text{profit}_d &= \text{netRevenue}_d - \text{ingredientCost}_d - \text{employeePay}_d. \end{aligned}$$

Waste cost is already part of ingredient cost, so subtracting it again would double count. It is tracked separately to explain lost margin. This modeled profit excludes rent, utilities, tax, equipment depreciation, and any other cost for which the stream has no event. Adding such events extends the equation rather than changing the meaning of existing fields.

3.2 Money by day is the primary XY plot

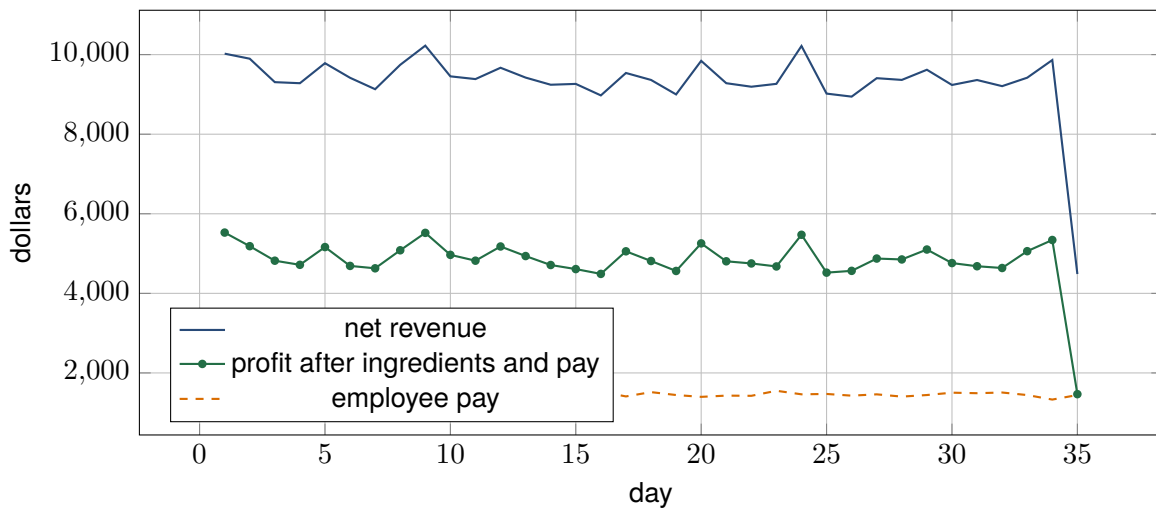


Figure 3.1: Daily bakery economics reduced from events. Day 35 is a partial day in the corpus, which explains its lower revenue and profit.

The employee-pay line is not a constant assumption. Each `ShiftPaid` event carries employee, role, minutes, hourly rate, and calculated pay. This lets the same chart answer counterfactual questions after a staffing event stream is changed.

3.3 Waste explains part of the margin

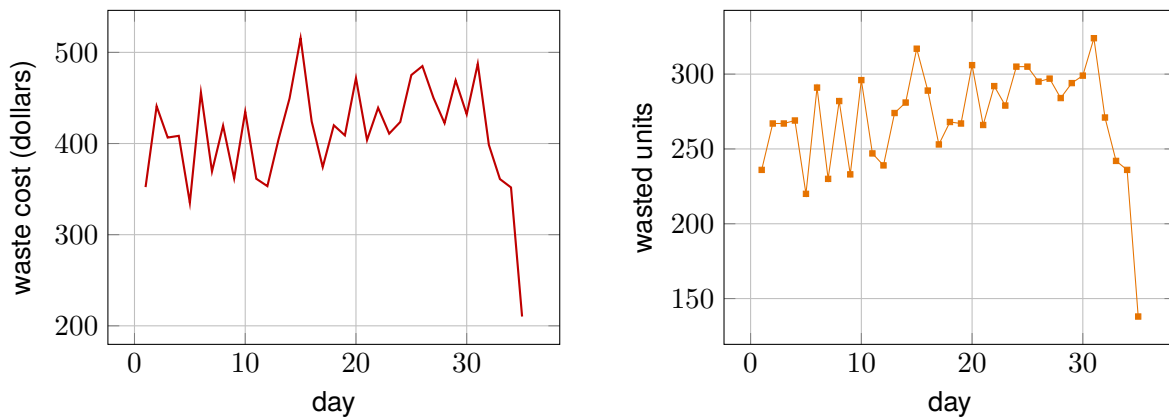


Figure 3.2: Waste cost and units by day, derived from failed bakes, cancellations, and failed deliveries in the same event stream.

Cost and units are deliberately separate. A wasted sourdough loaf and a wasted croissant do not have the same economic effect. Grouping the same events by product, outcome, employee shift, or hour of day produces further XY datasets without changing the scenario.

3.4 XY plots as questions over scenarios

The XY chart is not restricted to time on the horizontal axis. The reducer also generates daily order count against daily profit:



Figure 3.3: Daily order volume against profit. Each point is one day reduced from the event corpus.

The same mechanism can measure many scenarios: offered load against latency, staffing hours against profit, price against cancellation probability, queue capacity against completed orders, or waste against bake batch size. In each case the axes are named reductions over event streams. A parameter sweep produces several streams; observed operation supplies streams from different days. The chart renderer need not know which kind it received.

The corpus currently reports 16,099/20,000 delivered orders, median terminal latency 4,821,533 ms, 95th-percentile latency 7,863,807 ms, and a maximum of 46 active orders. These values are generated from the checked-in stream and serve as the default probability and performance inputs for the bakery model.

3.5 AI rendering from durable requirements to pseudocode

The grammar is useful for more than recognizing a saved corpus. It generates legal traces, so the same durable specification can drive software generation. The inputs are files, not the conversation that produced them. Let R be the complete current `Requirements.lean`, I the complete current `Implementation.lean`, P the protobuf schema, and C_k an optional existing implementation. Code generation has two forms:

$$(R, I, P) \longrightarrow C_0, \quad (R, I, P, C_k) \longrightarrow C_{k+1}.$$

The first creates software from scratch. The second revises existing software against the same stable inputs, retaining conforming code and replacing code that conflicts. Prior chat messages are deliberately absent from both functions. If a necessary decision is not present in R , I , or P , the generator reports the missing durable fact instead of recovering it from a long conversation.

For the bakery, the selected implementation backend is Go. The generated code uses Go protobuf bindings, deterministic `proto.Marshal`, `proto.Unmarshal`, and bounded channels. Requirement

comments are emitted beside the symbols and statements they justify:

```
// Code generated from Requirements.lean, Requirements.proto, and
// Implementation.lean. DO NOT EDIT.
package bakery

import (
    "context"
    "fmt"
    "time"

    "google.golang.org/protobuf/proto"
    pb "example.com/bakery/generated/bakerypb"
)

// Requirement: resource:Storefront
// Justification: each Storefront instance has finite inbound and outbound queues;
// a send to a full queue blocks and a receive from an empty queue blocks.
type Storefront struct {
    ID          string
    PaymentOut  chan []byte
    BakeryOut   chan []byte
    ResultsIn   chan []byte
    CustomerOut chan []byte
    Now         func() time.Time
}

// Requirement: resource:Storefront
// Justification: instantiate the capacities selected by Implementation.lean.
func NewStorefront(id string, paymentCapacity, bakeryCapacity, resultCapacity, customerCapacity int) *
    Storefront {
    return &Storefront{
        ID:          id,
        PaymentOut:  make(chan []byte, paymentCapacity),
        BakeryOut:   make(chan []byte, bakeryCapacity),
        ResultsIn:   make(chan []byte, resultCapacity),
        CustomerOut: make(chan []byte, customerCapacity),
        Now:         time.Now,
    }
}

// Requirement: proof:scenario protobuf round trip
// Justification: every generated event must resolve to bytes and be recoverable.
func writeProto(ctx context.Context, dst chan<- []byte, event *pb.ScenarioEvent) error {
    wire, err := proto.MarshalOptions{Deterministic: true}.Marshal(event)
    if err != nil {
        return fmt.Errorf("marshal scenario event: %w", err)
    }
    select {
    case dst <- wire: // Blocking bounded-channel send.
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}

// Requirement: proof:scenario protobuf round trip
// Justification: received bytes recreate the typed event used by the grammar.
func readProto(ctx context.Context, src <-chan []byte) (*pb.ScenarioEvent, error) {
    select {
    case wire := <-src: // Blocking receive from an empty channel.
        var event pb.ScenarioEvent
        if err := proto.Unmarshal(wire, &event); err != nil {
            return nil, fmt.Errorf("unmarshal scenario event: %w", err)
        }
        return &event, nil
    }
}
```

```

        case <-ctx.Done():
            return nil, ctx.Err()
    }
}

// Requirement: task:fulfill_order
// Justification: implement the generated legal production beginning at OrderPlaced.
func (s *Storefront) HandleOrderPlaced(ctx context.Context, placed *pb.ScenarioEvent) error {
    order, ok := placed.Message.Atom.(*pb.RequirementEnvelope_OrderPlaced)
    if !ok {
        return fmt.Errorf("expected OrderPlaced, got %T", placed.Message.Atom)
    }

    // Requirement: grammar:payment_parallel_bake
    // Justification: payment and baking fork from the same OrderPlaced event.
    paymentRequest := &pb.ScenarioEvent{
        Id: placed.Id + ".payment-requested", Prior: []string{placed.Id},
        Session: placed.Session, Task: placed.Task,
        Src: s.ID, Dst: "payment-1", TimeAt: unixMilliseconds(s.Now()),
        Message: &pb.RequirementEnvelope{Atom: &pb.RequirementEnvelope_PaymentRequested{
            PaymentRequested: &pb.PaymentRequested{PriceCents: order.OrderPlaced.PriceCents},
        }},
    }

    bakeRequest := &pb.ScenarioEvent{
        Id: placed.Id + ".bake-requested", Prior: []string{placed.Id},
        Session: placed.Session, Task: placed.Task,
        Src: s.ID, Dst: "bakery-1", TimeAt: unixMilliseconds(s.Now()),
        Message: &pb.RequirementEnvelope{Atom: &pb.RequirementEnvelope_BakeRequested{
            BakeRequested: &pb.BakeRequested{
                Product: order.OrderPlaced.Product, Quantity: order.OrderPlaced.Quantity,
            },
        }},
    }

    sendErrors := make(chan error, 2)
    go func() { sendErrors <- writeProto(ctx, s.PaymentOut, paymentRequest) }()
    go func() { sendErrors <- writeProto(ctx, s.BakeryOut, bakeRequest) }()
    for range 2 {
        if err := <-sendErrors; err != nil {
            return err
        }
    }

    results := make(map[string]*pb.ScenarioEvent, 2)
    for len(results) < 2 {
        result, err := readProto(ctx, s.ResultsIn)
        if err != nil {
            return err
        }
        if result.Session != placed.Session || result.Task != placed.Task {
            return fmt.Errorf("result dispatcher delivered the wrong (session, task)")
        }
        switch result.Message.Atom.(type) {
        case *pb.RequirementEnvelope_PaymentAuthorized,
             *pb.RequirementEnvelope_PaymentDeclined:
            results["payment"] = result
        case *pb.RequirementEnvelope_BakeCompleted,
             *pb.RequirementEnvelope_BakeFailed:
            results["bake"] = result
        }
    }

    paymentResult := results["payment"]
    bakeResult := results["bake"]
    accepted := isPaymentAuthorized(paymentResult) && isBakeCompleted(bakeResult)

```

```

// Requirement: grammar:payment_and_bake_join
// Justification: OrderAccepted/Rejected is enabled only after the two-of-two join.
decision := &pb.ScenarioEvent{
    Id: placed.Id + ".decision",
    Prior: []string{paymentResult.Id, bakeResult.Id, placed.Id},
    Session: placed.Session, Task: placed.Task,
    Src: s.ID, Dst: placed.Src, TimeAt: unixMilliseconds(s.Now()),
    JoinRequired: 2, JoinTotal: 2,
    JoinSelected: []string{paymentResult.Id, bakeResult.Id},
    Message: decisionMessage(accepted),
}
return writeProto(ctx, s.CustomerOut, decision)
}

// Requirement: performance:client-experienced byte rate
// Justification: pool work over the observation time experienced by each client.
func ClientExperiencedRate(samples []CompletedWork) (bytes, milliseconds uint64) {
    for _, sample := range samples {
        bytes += sample.BytesMoved
        milliseconds += sample.End.TimeAt - sample.Start.TimeAt
    }
    return bytes, milliseconds
}

// Requirement: performance:server aggregate byte throughput
// Justification: measure total work over one server wall-clock window.
func ServerAggregateRate(samples []CompletedWork) (bytes, milliseconds uint64) {
    start, end := samples[0].Start.TimeAt, samples[0].End.TimeAt
    for _, sample := range samples {
        bytes += sample.BytesMoved
        if sample.Start.TimeAt < start { start = sample.Start.TimeAt }
        if sample.End.TimeAt > end { end = sample.End.TimeAt }
    }
    return bytes, end - start
}

type CompletedWork struct {
    Start, End *pb.ScenarioEvent
    BytesMoved uint64
}

func unixMilliseconds(now time.Time) uint64 { return uint64(now.UnixMilli()) }

// The following helpers are generated by exhaustive matches over grammar atoms.
func isPaymentAuthorized(*pb.ScenarioEvent) bool
func isBakeCompleted(*pb.ScenarioEvent) bool
func decisionMessage(bool) *pb.RequirementEnvelope

```

The pseudocode does not become a second source of requirements. It is discarded and regenerated when R or I changes. Generated source code is likewise a projection, except that revision mode accepts the current code as an additional input so hand-written conforming sections need not be destroyed. In both modes, the grammar supplies positive tests by generating legal traces and negative tests by rejecting mutations outside its language. Byte round trips then check that the implementation and its transport bindings preserve the observable scenario.

Every generated code unit carries a machine-readable requirement reference and a short justification beside the code it explains. The reference points into the durable requirement namespace, not to a chat turn. Generation also emits a traceability manifest mapping requirement IDs to source files, symbols, and tests. An implementation mapping without a requirement reference is an orphan and fails implementation-plan validation; generated-source validation must apply the same rule to emitted symbols. Several code units may cite one requirement, and one code unit may cite several

requirements; the mapping is many-to-many.

Chapter 4

A Multiparty Grammar of Behaviour

A protocol is a language whose terminal symbols are visible events. A production describes a legal continuation. The context-free grammar is the visual and conceptual starting point, not a claim that every useful protocol language is context-free.

4.1 A small authoring algebra

```
inductive Grammar (Event : Type) where
| empty
| event  : Event -> Grammar Event
| seq    : Grammar Event -> Grammar Event -> Grammar Event
| choice : List (Grammar Event) -> Grammar Event
| par    : Grammar Event -> Grammar Event -> Grammar Event
| where_ : (List Event -> Bool) -> Grammar Event -> Grammar Event
```

`seq` preserves time order. `choice` exposes alternatives rather than burying them in control flow. `par` admits interleavings while preserving the internal order of each side. A `where_` clause checks a fact accumulated from the prior trace and provides the escape from context-free structure when genuine context sensitivity is needed.

A choice does not need a separate annotation naming who makes it. Each branch eventually resolves to terminal events, and each terminal resolves to framed bytes with a source and destination. The first observed terminal that distinguishes two branches identifies both the selected branch and the actor that emitted the distinguishing bytes. If two branches resolve to the same observable bytes, then at this abstraction they are not distinct choices and should be merged. This removes an otherwise ambiguous second account of agency from the grammar.

Simple finite conversations can compile to regular languages. Nested request structure motivates context-free productions. Correlation, freshness, uniqueness, and authorization typically require parameter flow or guarded productions.

4.2 A BNF for composable interactions

The authoring notation can be presented independently of Lean as a small BNF. Named productions are interactions; referring to a production composes that interaction into a larger one.

```

<specification> ::= <production>
                  | <production> <specification>

<production>    ::= <name> "[:=" <expression>

<expression>    ::= "empty"
                  | <event>
                  | <name>
                  | "(" <expression> ";" <expression> ")"
                  | "(" <expression> "|" <expression> ")"
                  | "(" <expression> "||" <expression> ")"
                  | "repeat" "(" <expression> ")"
                  | "join" "(" <natural> "of" <expressions> ")"
                  | "when" "(" <predicate> "," <expression> ")"
                  | "after" "(" <name> "," <expression> ")"

<event>         ::= <actor> "->" <actor> ":"
                  <message> "(" <fields> ")"

<fields>        ::= ""
                  | <field>
                  | <field> "," <fields>

<field>         ::= <name> "=" <value>

```

The operators have the same meanings as the grammar AST: semicolon is sequence, vertical bar is alternative, and double vertical bar is parallel composition by order-preserving interleaving. `after(P,E)` includes `E` only on a branch in which production `P` completed. Predicates and repeated field variables supply the context-sensitive correlation that plain context-free syntax does not express by itself.

A threshold join is explicit rather than inferred from the number of observed predecessors:

```

firstTwoReplicas ::= join(2 of replicaA, replicaB, replicaC)
firstTwoThenReply ::= (firstTwoReplicas ; reply)

```

The continuation becomes enabled after any two of the three branches complete. Its first emitted event records `joinRequired=2`, `joinTotal=3`, and exactly two selected branch-terminal IDs. Those IDs also occur in the event's ordinary prior list, which may contain other backpointers such as the task start. The threshold declaration says why two predecessors suffice; the selected IDs say which two satisfied this instance. Unselected branches are not implicitly canceled and may still emit events. Cancellation, if required, must be an explicit message or production.

The OAuth example decomposes into a collection of independently readable interactions:

```

applicationRequestAndRedirect ::=
  (Browser -> Application : ApplicationRequest(resource=resource) ;
   Application -> Browser : AuthorizationRedirect(
     state=state, redirectUri=callback))

authorizationGranted ::=
  (Authorization -> Browser : AuthorizationCodeReturned(
    state=state, code=code) ;

```

```

    Browser -> Application : AuthorizationCallback(
        state=state, code=code))

authorizationDenied ::=
    (Authorization -> Browser : AuthorizationErrorReturned(
        state=state, reason=access_denied) ;
    Browser -> Application : AuthorizationErrorCallback(
        state=state, reason=access_denied))

authorizeAndDecide ::=
    (Browser -> Authorization : AuthorizationRequest(
        state=state, redirectUri=callback) ;
    (authorizationGranted | authorizationDenied))

exchangeAndFetch ::=
    (Application -> Authorization : TokenRequest(code=code) ;
    Authorization -> Application : AccessTokenIssued(token=token) ;
    Application -> Resource : ResourceRequest(token=token) ;
    Resource -> Application : ResourceReturned(profile=profile) ;
    Application -> Browser : ApplicationResponse(profile=profile))

oauth ::=
    (applicationRequestAndRedirect ;
    authorizeAndDecide ;
    after(authorizationGranted, exchangeAndFetch))

```

This form exposes two useful levels without defining two protocols. At the higher level, `oauth` is three production references. Expanding a reference reveals its events and nested alternatives. The same productions can be reused in another system, placed in parallel with independent interactions, guarded, or repeated. Composition operates on their accepted event languages; it is not textual copying.

4.3 Parameters carry information

A production should bind values at one event and reuse them later. Conceptually:

```

def login (browser server provider : Actor) (state : OAuthState) : Grammar Event :=
    event (redirect server browser provider state) >>>
    event (authorize browser provider state) >>>
    callback browser server state

```

The repeated state is not decorative. It states the information-flow relationship that distinguishes a matching callback from an unrelated one. Lean's types, constructors, and ordinary function parameters are the surface language; the fixed library supplies grammar composition and semantics.

4.4 Composition without omniscience

A participant's implementation need not know the global execution state. The global grammar states the contract, while projection derives each participant's view. Composition should preserve

independently specified modules and make shared events explicit. Proving useful projection and composition theorems is a core research obligation, not an implementation footnote.

4.5 Actors multiplex bounded conversations

An actor has separate finite-capacity inbound and outbound message queues. Queue capacity is part of the observable execution model: producing a message may block when the outbound queue is full, and transport may block when the destination's inbound queue is full. Sending is therefore distinct from delivery and from the destination actor consuming the message.

Messages carry a session identifier as well as a task identifier. Their pair is the correlation key for actor-local work:

```
structure WorkKey where
  session : SessionId
  task    : TaskId

structure ActorRuntime where
  inboundCapacity : Nat
  outboundCapacity : Nat
  inbound         : List Envelope
  outbound        : List Envelope
  inFlight        : List (Prod WorkKey GrammarResidual)
  inboundBounded  : inbound.length <= inboundCapacity
  outboundBounded : outbound.length <= outboundCapacity
```

An actor may have zero or many distinct `WorkKeys` in flight. Consuming an inbound envelope selects or creates the residual associated with that envelope's `(session, task)` pair; it does not replace one actor-wide “current task.” Messages from different keys may interleave while the grammar preserves the order required within each conversation.

The queues have Go-channel-like operational semantics. Capacity is enforced when an operation is attempted, not checked only after constructing a new state:

```
send(channel, message) =
  if channel.length < channel.capacity
  then sent(channel ++ [message])
  else blocked

receive(channel) =
  if channel = message :: rest
  then received(message, rest)
  else blocked

tryReceive(channel) =
  if channel = message :: rest
  then some(message, rest)
  else none
```

A blocked operation makes no progress and leaves the channel unchanged. A blocking actor may run again when the required space or data becomes available. `tryReceive` is a different operation: `none` reports that the channel is empty now and lets the actor continue. It does not put the actor to sleep, and it must not be confused with receiving a valid optional payload.

The resulting transitions are explicit:

- an actor step appends a produced envelope to its outbound queue;
- a transport step moves the outbound head to the destination's inbound queue when both operations are within capacity;
- a receive step removes an inbound envelope and advances the matching (`session`, `task`) residual;
- sending to a full outbound queue blocks the sender without enqueueing;
- transport to a full inbound queue blocks without removing the outbound head;
- blocking receive on an empty inbound queue blocks, while nonblocking `tryReceive` returns `none` and permits later work.

Queue order and predecessor order are not identical. FIFO determines which queued envelope is available next, while `priorIds` records the event-order graph, including forks and joins across actors. Incomparable branches may commute; the pointer relation does not itself claim that one event caused another.

Chapter 5

The Scenario Authoring Language

This chapter proposes the first concrete surface language. It is intentionally a small Lean DSL: ordinary `def`, function parameters, `let`, typed values, and `do` notation provide the host language; a fixed library adds `scenario`, `endpoint`, `choice`, `probability`, and `rendering` operations. The examples are a design target. Names may change as the Lean implementation makes their semantics precise.

5.1 The smallest useful scenario

A scenario is a named grammar fragment. Every emitted terminal is an observable message event. Endpoint commands establish the `from` and `to` context for the messages that follow.

```
def requestQuote (customer shop : Actor) : Scenario :=
  scenario "request a quote" do
    let requestId <- declare RequestId "requestId"

    from_ customer
    to_ shop
    emit "quote requested" [field "requestId" requestId]

    let quoteId <- declare QuoteId "quoteId"
    let amount <- declare Money "amount"

    from_ shop
    to_ customer
    emit "quote offered"
      [ field "requestId" requestId
        , field "quoteId" quoteId
        , field "amount" amount
      ]
```

The spelling `from_` avoids collision with Lean syntax while remaining visually explicit. Endpoint commands affect authoring context only; elaboration copies both endpoints into every event. A scenario with an `emit` before both endpoints have been established is rejected.

5.1.1 Variables and assignment

There are two kinds of assignment.

- Ordinary `let x := expression` evaluates a known Lean expression.
- `let x <- declare T "x"` introduces a symbolic value of type `T`. The grammar binds it when generating or matching a trace, and later events may reuse it.

Variables are immutable. A later `let attempt := attempt + 1` creates a new binding rather than mutable hidden state. Information flow is consequently visible in the grammar's data dependencies. Equality of two occurrences of `requestId` means they carry the same value, not merely values with the same field name.

A value received from the environment is bound by matching an observable event:

```
from_ customer
to_ shop
receive "quote accepted" do
  let quoteId <- take QuoteId "quoteId"
  let payment <- take PaymentToken "paymentToken"

from_ shop
to_ bank
emit "payment authorization requested"
  [field "quoteId" quoteId, field "paymentToken" payment]
```

`receive` and `emit` both denote grammar terminals; they differ only in the viewpoint used when projecting a global scenario to executable participant code.

5.2 Vague messages first, typed alphabets later

Early requirements should not require a finished protobuf schema. A vague message has a stable semantic name and a list of named, typed fields. It is already precise enough to express ordering, correlation, and temporal properties.

```
emit "quote offered"
  [field "requestId" requestId, field "amount" amount]
```

Later, a refinement maps that terminal to a concrete alphabet:

```
refineMessage "quote offered" with Proto.Shop.QuoteOffered where
  request_id := field "requestId"
  quote_id   := field "quoteId"
  cents      := encodeMoney (field "amount")
```

The refinement obligation is language inclusion: every concrete implementation trace, after erasing representation details, must be admitted by the abstract grammar. Refinement may decide serialization, retry mechanics, and internal state; it must not invent externally visible behaviour forbidden by the requirements.

5.3 One concern per scenario

Scenarios should be small enough to read as stories and independent enough to render separately. The quote protocol can be decomposed as follows.

```

def quoteAccepted (customer shop : Actor) : Scenario :=
  scenario "quote accepted" do
    include (requestQuote customer shop)
    -- bind quoteId from the included scenario's exports
    use quoteId <- exported QuoteId "quoteId"
    from_ customer; to_ shop
    emit "quote accepted" [field "quoteId" quoteId]

def quoteDeclined (customer shop : Actor) : Scenario :=
  scenario "quote declined" do
    include (requestQuote customer shop)
    use quoteId <- exported QuoteId "quoteId"
    from_ customer; to_ shop
    emit "quote declined" [field "quoteId" quoteId]

def quoteOutcome (customer shop : Actor) : Scenario :=
  oneOf "customer decision"
  [ quoteAccepted customer shop
    , quoteDeclined customer shop
  ]

```

`include` is textual only in the rendered book view; semantically it is a reference to a named nonterminal. The compiler reads all referenced scenarios, checks exported bindings, detects cycles, and builds one grammar. Tools retain the scenario boundary for navigation, diagrams, tests, and counterexamples.

5.4 Interaction diagram: accepted quote

The first rendering of any scenario is a sequence-style interaction diagram. It shows observable messages and parameter flow without suggesting an internal implementation architecture.

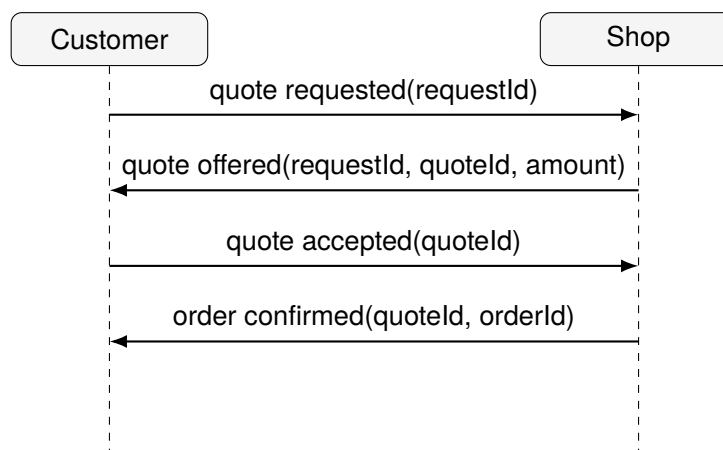


Figure 5.1: Interaction diagram derived from the accepted-quote scenario.

5.5 Choices and unresolved probability

Nondeterminism and probability are different. `oneOf` says that multiple behaviours are legal without claiming how often they occur. `weighted` attaches a distribution when the abstraction has one.

```
def paymentOutcome : Scenario :=
  oneOf "payment outcome" [paymentApproved, paymentRejected]

def estimatedCustomerDecision : Scenario :=
  weightedOneOf "customer decision"
  [ weighted (95 / 100) customerSubmitsPayment
    , weighted ( 5 / 100) customerAbandonsPayment
  ]
```

There is no separate `choose customer` annotation. The branch is resolved when its distinguishing terminal becomes bytes: the terminal names its source, so the trace already shows that the customer emitted the event that selected the branch. Probabilities belong to branches, not to an extra, potentially conflicting claim about a chooser. A deterministic service that consumes probabilistic user input remains part of the induced probabilistic system: its probability-one transitions carry the incoming path mass forward. An early requirement can use `oneOf`, a symbolic weight, or a range such as `between 0.90 0.99`. Implementation refinement resolves those claims to a well-defined stochastic model or to deterministic conditions. Refinement must preserve possibility even when a current implementation assigns probability zero to a legal abstract branch; otherwise changing operational policy would silently change the contract.

Two branches may share a prefix. In that case the choice remains unresolved through the common terminals and resolves only at the first different byte-level terminal. Branches whose complete terminal byte languages overlap are observationally ambiguous at that point; the requirement must merge them or add an observable discriminator if downstream actors need to distinguish them.

5.6 Actor specifications and instances

An actor name in a grammar denotes a reusable specification, not necessarily one process. A finite population gives that specification concrete identities:

```
actor Client
  |>.instances "client-" 20 (sticky "session")

actor Server
  |>.instances "server-" 2 (shardHash "tenant_id")
```

The grammar may say `Client -> Server`; a generated scenario resolves those roles to instances such as `client-7 -> server-2`. Each instance has its own inbound and outbound queues and its own set of in-flight (`session, task`) pairs, while inheriting behavior and resource bounds from the actor specification. A population is a homogeneous service cluster: every member must implement that one actor specification. A cluster cannot quietly mix gateways, databases, and workers merely because they share a deployment boundary.

The cluster also declares how a message selects an instance. `sticky` keeps the value of an observable affinity field on the same member; `roundRobin` cycles through members in a declared

order; and `shardHash` hashes an observable shard-key field. This decision changes the legal per-instance histories and therefore queue pressure, hot spots, failure impact, and replay. Kafka partition keys, MongoDB shard keys, and DynamoDB partition keys motivate the shard-hash abstraction, although each real system adds product-specific replication and routing semantics beyond this small model. Population size is finite and explicit so queue, memory, probability, and temporal checks still have a bounded state space.

5.7 Non-functional rates and scalability

Timing requirements are stated over boundary messages rather than intervals hidden inside an event. Suppose every client repeatedly creates a fresh server task. Each completed unit records a start message, an end message that backpoints to that start, and a nonnegative amount of work. If one moved byte is one work unit and timestamps are milliseconds, an exhaustive observation list supports two different rates.

The client-experienced, properly weighted rate is

$$X_{\text{client}}(N) = \frac{\sum_i \text{work}_i}{\sum_i (e_i.\text{timeAt} - s_i.\text{timeAt})}.$$

This is the duration-weighted mean of the individual observed rates. Long requests therefore contribute in proportion to how long a client experienced them. It is not the unweighted mean of per-request ratios.

The aggregate server throughput is instead

$$X_{\text{server}}(N) = \frac{\sum_i \text{work}_i}{\max_i (e_i.\text{timeAt}) - \min_i (s_i.\text{timeAt})}.$$

Its denominator is one wall-clock observation window, so overlapping client waits are counted once. The two quantities answer different questions and must not share a metric name. Plotting either explicitly chosen rate against the number of active client instances yields observations that can be fitted to the Universal Scalability Law,

$$X(N) = \frac{\gamma N}{1 + \alpha(N - 1) + \beta N(N - 1)}.$$

Gunther's *Guerrilla Capacity Planning* gives the practical capacity-planning setting for this law and develops it as the Universal Law of Computational Scaling [4]. The point is operational, not decorative: a requirements argument should expose contention, coherency, saturation, and retrograde scaling before the first production deployment supplies those facts by falling over.

The event list supplies the observations; estimating γ , α , and β is a separate statistical step. The list must be exhaustive for the declared observation window, or omitted work and unfinished tasks bias both rates.

5.7.1 The three USL regimes and capacity

The scale parameter γ is analogous to clock speed: it is the single-client service-rate scale and multiplies the entire curve vertically. It is not literally a processor clock, because an observed unit of work may include networks, storage, and several actors. Changing γ does not move the population at which the dimensionless USL curve peaks.

The two overhead parameters have distinct effects:

$$\begin{aligned}
 \alpha = 0, \beta = 0 : \quad X(N) &= \gamma N, & \text{ideal linear scaling,} \\
 \beta = 0 : \quad X(N) &= \frac{\gamma N}{1 + \alpha(N - 1)}, & \text{contention-limited saturation,} \\
 \beta > 0 : \quad X(N) &= \frac{\gamma N}{1 + \alpha(N - 1) + \beta N(N - 1)}, & \text{coherency-limited retrograde scaling.}
 \end{aligned}$$

For $0 \leq \alpha < 1$ and $\beta > 0$, differentiation gives the finite capacity population

$$N_* = \sqrt{\frac{1 - \alpha}{\beta}}.$$

The observed integer capacity is one of the adjacent integer populations, whichever has the larger measured or fitted throughput. When $\beta = 0$ and $\alpha < 1$, the curve increases toward the asymptote γ/α but has no finite USL peak. When both parameters are zero, the mathematical model has no internal capacity limit; a real finite system must eventually reveal an omitted bottleneck or cease to fit that ideal case.

Figure 5.2 uses normalized throughput X/γ . The fitted example uses $\alpha = 0.1$ and, in the third panel, $\beta = 0.005$, giving $N_* \simeq 13.42$. Throughput to the right of the marked point declines even as more clients are added.

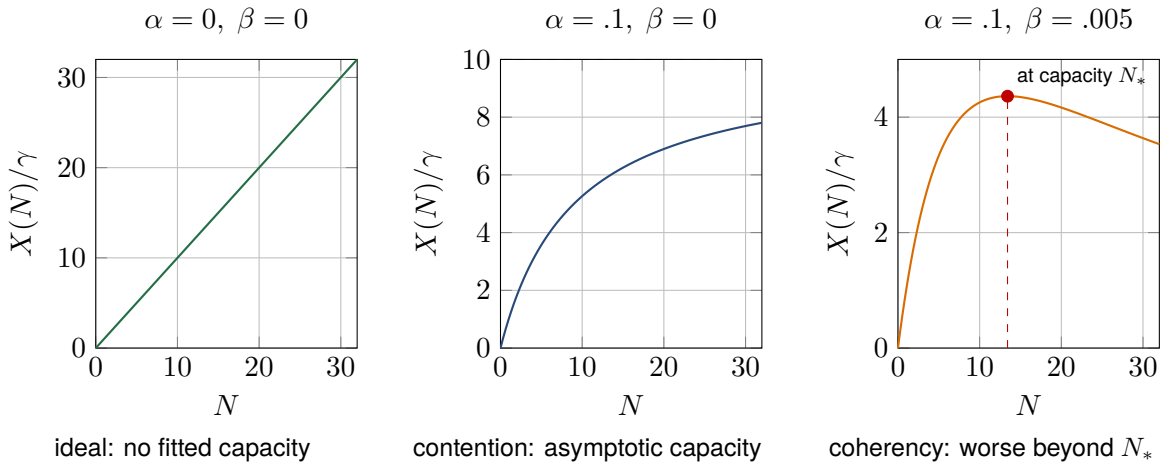


Figure 5.2: Universal Scalability Law regimes. γ rescales every vertical axis. Contention alone saturates; coherency creates a finite peak and declining throughput beyond capacity.

Population capacity is not the only overload boundary. For a fixed deployment with sustainable service rate μ , offered load λ is at capacity when $\lambda = \mu$. Below capacity, a simple queueing illustration has response time proportional to $1/(\mu - \lambda)$ and queued work proportional to $\lambda/(\mu - \lambda)$. At and above capacity these steady-state quantities are unbounded: completed throughput is capped by μ , while unfinished work, latency, queued bytes, and memory pressure accumulate. A finite queue converts that growth into blocking or rejection; exhausting the memory budget is a fatal terminal event rather than another slow sample.

For the queueing foundation behind this illustration—including workload characterization, service demand, utilization, response time, queueing networks, and cost/performance tradeoffs—see

Menascé and Almeida's *Capacity Planning for Web Services* [7]. USL and a single queue answer different questions and belong together: USL summarizes scaling across populations, while queueing models explain how offered work waits for finite service centers. Neither rescues a requirements model whose demand, service rates, queue bounds, observation window, and overload behavior were never stated.

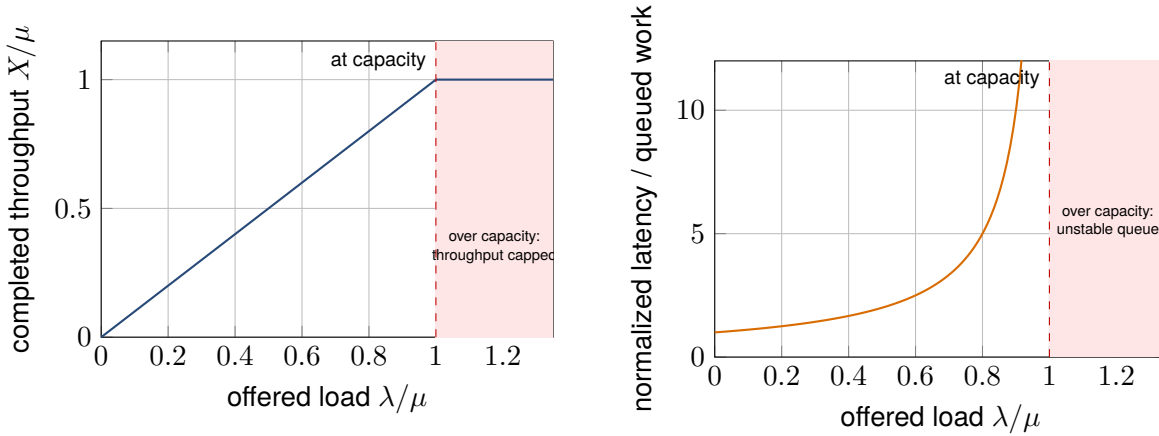


Figure 5.3: Offered-load capacity. Beyond $\lambda/\mu = 1$, throughput cannot increase but queued work and latency continue to worsen until backpressure, rejection, an outage, or fatal memory exhaustion intervenes.

This is why performance belongs in requirements rather than in a post-deployment incident report. Before implementation, the specification should make it possible to ask whether forecast demand fits below sustainable capacity, whether latency and queue bounds remain acceptable, what happens to rejected or blocked work, how many replicas are affordable, and which failure modes are fatal. The goal is not to predict every scheduler tick. It is to rule out designs that work only at toy load and collapse the first time real users arrive.

5.8 Outages, recovery, and replica populations

Reliability assumptions belong to actor specifications and are inherited by their instances. A contract may set an outage probability and an expected mean time to repair (MTTR):

```
actor Server
  |>.instances "server-" 2 (roundRobin)
  |>.outageProbability (20 / 10000)
  |>.meanTimeToRepair 45.seconds
```

These are specified assumptions. Measurements come from separate `Unavailable` and `Recovered` messages. The recovery message backpoints to the unavailable message, so one repair duration is `recovered.timeAt - unavailable.timeAt`. For R replicas observed from t_0 through t_1 , the observed outage fraction and MTTR are

$$\hat{O} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{R(t_1 - t_0)}, \quad \widehat{MTTR} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{|incidents|}.$$

Availability over the same complete window is $1 - \hat{O}$. Both values can be plotted against time, replica count, offered load, or a deployment policy. This supports Kubernetes-style experiments in which pods are actor instances, restarts are recovery events, and changing replica populations yields different reliability and throughput observations. A configured outage probability or MTTR is not silently replaced by its observed estimate; comparing them is the check.

5.9 Infrastructure cost and demand-constrained supply

A running service has at least two distinct time-based costs. Hardware is paid for while an instance is provisioned, whether or not the service is healthy. The service may also have a license, managed-service, or operational cost for each hour it is available. Those are not task-event costs and must not be multiplied by request count:

```
cost Server
  |>.hardwarePool "general-purpose"
  |>.hardwareCostPerProvisionedHour 12 USD.cent
  |>.serviceCostPerAvailableHour 3 USD.cent
  |>.sustainableCapacityPerHour 3600000 bytes
```

For R replicas, provisioned hours H_r , available hours A_r , hardware rate c_h , and service rate c_s , the accounting-period infrastructure cost is

$$C(R) = \sum_{r=1}^R (H_r c_h + A_r c_s).$$

Supply planning is a constrained optimization, not merely “pick the cheapest server.” Let one replica sustain μ work units per hour and let a be the required availability fraction. A candidate replica count is feasible for demand D only when

$$R\mu a \geq D.$$

With an ideally balanced routing policy, the nominal load on each server is D/R . Spawning more homogeneous servers therefore lowers average per-server load. That statement alone cannot choose a replica count: every added server also adds provisioned hardware cost and, while healthy, service cost. The useful graph places D/R , observed load for each concrete instance, effective supply, and total hourly cost on the same replica-count axis.

LeanFM enumerates the declared finite set of permitted replica counts, rejects infeasible points, and chooses the remaining point with minimum expected hourly cost $R(c_h + ac_s)$. The derived XY artifact plots demand, effective supply, and cost against replica count and marks the feasible optimum. It also reports headroom and rejected demand so an apparently cheap under-supplied plan cannot win. The average D/R is not a promise of balance. Routing policy matters here: shard hashing can create a hot shard even when aggregate supply exceeds demand, while sticky routing and round-robin make different balance assumptions. Product-specific replication, reserved pricing, autoscaling delay, and nonlinear volume tiers require additional declared constraints rather than silent arithmetic.

5.10 Specified dwell time

A specification may attach a dwell time to an event step or to a named production. The dwell is a duration on the same monotonic time scale as event clocks:

```
emit "token requested" [field "code" code]
  |>.dwell 3.milliseconds

include exchangeAndFetch
  |>.dwell 12.milliseconds
```

Taking a step with dwell d advances the model clock from t to $t + d$ before timestamping the event that completes the step. A production dwell constrains the elapsed time between its entry and exit boundary events; it does not assign the same dwell independently to every contained event. Exact values, bounded ranges, or weighted dwell outcomes may be used when the corresponding timing information is available.

Dwell is part of the specification, so it can drive latency, throughput, timeout, and queue-pressure calculations. It does not create event order: `priorIds` supplies the predecessor relation, while the grammar separately states why a sequence or dependency is legal.

5.11 Estimating completion from an event stream

The same specification can consume observed events. A terminal event of any kind names its corresponding start event in its backpointer list. The reducer follows that identity rather than guessing from adjacency or selecting the first open (`session`, `task`) attempt. Each matched pair yields a task attempt:

```
structure TaskAttempt where
  key      : WorkKey
  started  : Event
  completed : Option Event
  outcome  : Option TaskOutcome
```

```
def latency (attempt : TaskAttempt) : Option Duration :=
  attempt.completed.map fun completed =>
    completed.timeAt - attempt.started.timeAt
```

For an observation window containing N starts, C matched terminal events, S successful terminals, and completed latencies $L_1, \dots, L_C$, the basic estimates are

$$\hat{P}(\text{complete}) = \frac{C}{N}, \quad \hat{P}(\text{success} \mid \text{complete}) = \frac{S}{C}, \quad \hat{E}[L \mid \text{complete}] = \frac{\sum_i L_i}{C}.$$

Attempts without a terminal event remain *in flight*. They contribute to the number started and to queue/load measurements, but not to completed-task latency or to the conditional success denominator. Calling them failures would bias both probability and latency whenever the observation window ends before they do.

The reducer can group these sufficient statistics by task, actor, time bucket, maximum concurrency, or queue occupancy and feed them directly to pie, time-series, and XY plots. Specified probabilities and dwell times are therefore predictions; stream-derived estimates provide measurements against which those predictions can be checked.

In the executable reducer, an entry event opens a `TaskAttempt`, progress events leave it open, and a success or failure event closes the open attempt with the same `(session, task)` key. The resulting attempts feed the estimates above.

5.12 Composing requirements means intersecting constraints

Scenario composition builds behaviour in familiar language-theoretic ways:

Operator	Language operation	Purpose
then a b	concatenation	a happens before b
oneOf [a,b]	union	either story is legal
together a b	shuffle/interleaving	independent concurrent stories
repeat a	closure	repeated conversation
where_ p a	trace filtering	context-sensitive legality

Combining requirements is subtler. A requirement usually does not add another message sequence; it removes histories that violate a concern. Therefore `requireAll` denotes intersection of accepted trace sets.

```
def quoteSystem : SystemSpec :=
  generatedBy quoteOutcome
  |> requireAll
    [ correlate "offered quote matches request"
      (sameField "requestId")
    , forbid "confirmation without acceptance"
      (event "order confirmed" && notPreviously "quote accepted")
    , ctl "every accepted quote is answered"
      (AG (accepted --> AF (confirmed || rejected)))
    , timing "answer within service objective"
      (accepted --> within 2.seconds (confirmed || rejected))
    ]
```

This distinction prevents an important category error: union and concatenation assemble possible stories; intersection layers independent obligations over those stories. A system is consistent only if the intersection remains nonempty. When it is empty, tooling should return a minimal conflicting set of requirements and the shortest available counterexample prefix.

5.13 State-machine rendering

The same scenario grammar can be rendered as a state machine. States are grammar residuals—the legal continuations after a trace prefix—rather than snapshots of private implementation memory.

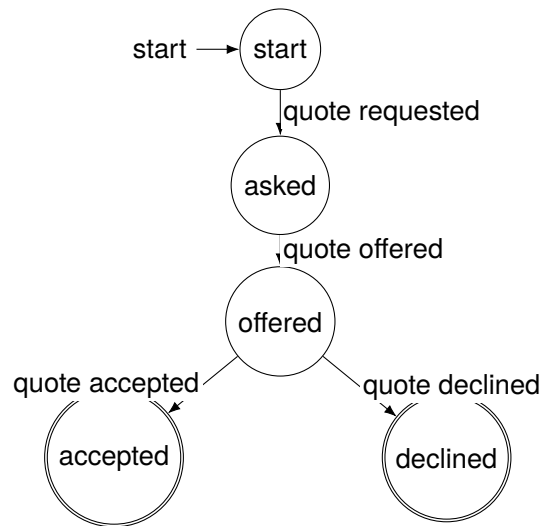


Figure 5.4: A state machine derived from grammar residuals.

5.14 Charts are declarative trace reductions

Charts are also specified in the Lean subset. A chart names an observable event source, a grouping or x expression, and an aggregate or y expression. It does not contain rendering code.

```

def outcomeShare : ChartSpec :=
  pie "quote outcomes" do
    source events
    where_ (kind == "quote accepted" || kind == "quote declined")
    groupBy kind
    value count

def responseLatency : ChartSpec :=
  line "quote response latency" do
    source (correlate events by "quoteId")
    x clock
    y (elapsed "quote offered" "customer decision")
    series "p50" median
  
```

```

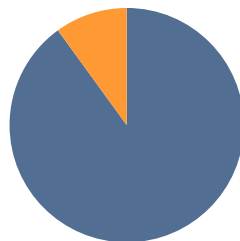
def latencyByConcurrency : ChartSpec :=
  xyplot "latency by concurrent sessions" do
    source completedRuns
    x maxConcurrentSessions
    y (mean (elapsed firstEvent terminalEvent))

def queueByOfferedLoad : ChartSpec :=
  xyplot "queue occupancy by offered load" do
    source completedRuns
    x requestsPerSecond
    y (mean totalQueueOccupancy)

def propagatedGrantProbability : ChartSpec :=
  xyplot "user probability propagated through deterministic servers" do
    source parameterSweep
    x userGrantProbability
    y terminalGrantProbability

```

An XY plot preserves two numeric axes. It differs from a time series, whose x -axis is specifically an event clock, and from a categorical chart, whose x values are labels. The first two examples reduce each completed model run to one point. The third sweeps a model parameter and checks that deterministic downstream processing transports the user's probability mass.



accepted 90%
declined 10%

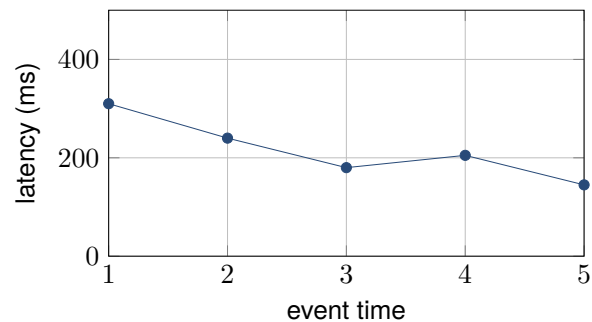


Figure 5.5: Example pie and line renderings from declarative chart specifications.

Interaction diagrams, state machines, and charts are projections of the same typed scenario and event definitions. Keeping them derived prevents a diagram from becoming a second, drifting specification.

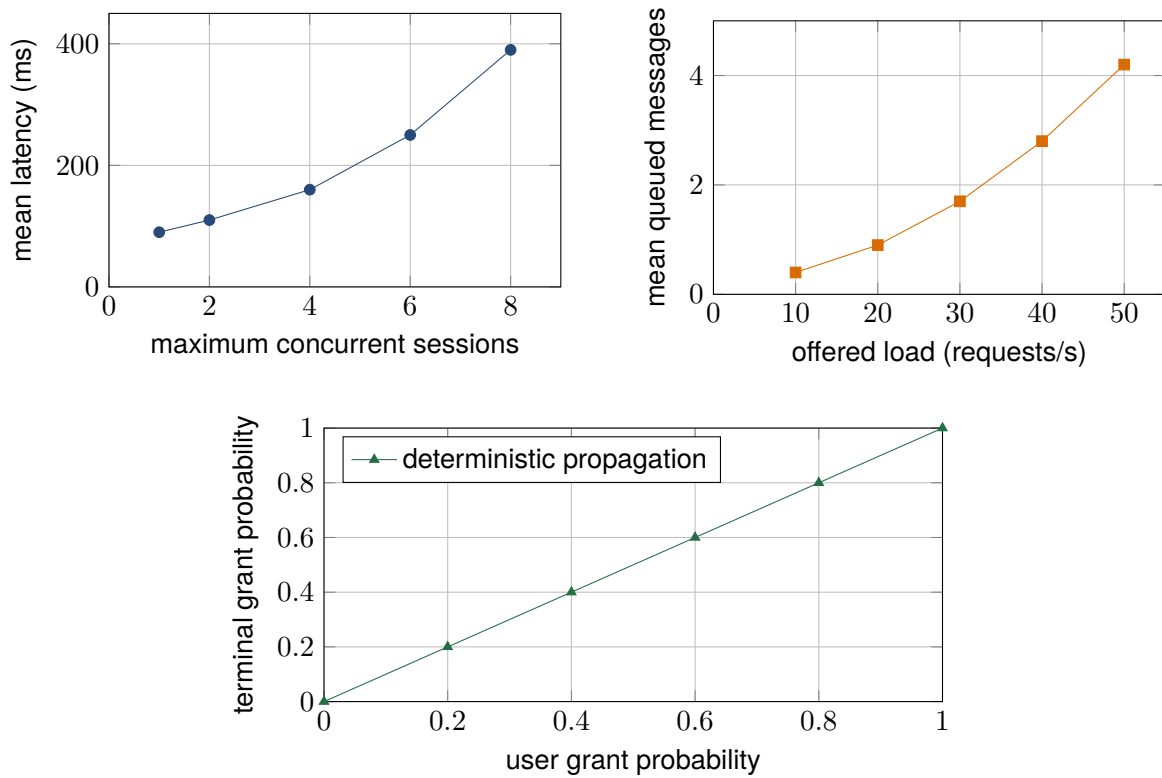


Figure 5.6: XY-plot examples. The values are illustrative model-run data: clocks and correlation produce latency, bounded queues produce occupancy, and probability-one server steps preserve user-originated grant mass.

Chapter 6

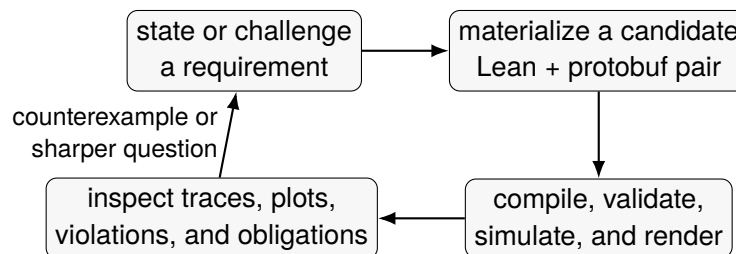
Arguing Requirements through the Lean Web Server

The LeanFM web server presents a conversational workbench, but conversation is not its product. Its product is a durable set of artifacts: a typed Lean requirements model, a protobuf description of the bytes exchanged by the participants, and a separate Lean implementation plan. The useful form of requirements vibecoding is therefore not “ask a model to write the system.” It is an argument in which a person keeps challenging a proposed boundary until the remaining claims can be compiled, checked, measured, or left as explicit proof obligations.

Chat history is evidence about how a decision was reached. It is not the decision. A future reader must be able to discard the transcript, open the generated files, and recover the actors, observable events, alternatives, correlation rules, timing assumptions, resource bounds, and properties.

6.1 The argument loop

The interface resembles ChatGPT because natural language is convenient for finding omissions. The loop, however, has a formal boundary:



The human contribution is often adversarial. “The upload completes” should prompt: what observable event constitutes completion, who observes it, and does it mean receipt, durable storage, or policy acceptance? “The service handles load” should prompt: how many messages fit in each queue, how many (session, task) pairs may be active, and what happens at the bound? The assistant’s job is to turn each answer into structure, not merely agreeable prose.

A productive exchange might proceed as follows.

Person: A submitted upload eventually completes.

Assistant: I will treat an `uploadCompleted` response as completion.

Person: No. Completion requires the storage actor's acknowledgment, and the client must be able to distinguish rejection from timeout. Both queues are finite.

Assistant: Then the grammar has accepted, rejected, and timed-out alternatives; acceptance contains a storage acknowledgment before the response. I will add queue capacities and a bounded-liveness obligation.

Person: Also show the behavior with many sessions in flight, estimate the completion distribution from an event stream, and fail the requirement if queued bytes exceed the memory budget.

Every turn should change a named assumption, artifact, check, or open obligation. If it changes none of them, it has not improved the specification.

6.2 The server is a staging environment

The executable `lake exe leanfm-server` serves the assistant workbench on the loopback interface. An authenticated session has its own generated `Requirements.lean`, `Requirements.proto`, and `Implementation.lean`. The first two specify observable behavior and values that resolve to bytes. They do not imply HTTP. The implementation file selects how those bytes travel—for example HTTP method and path, an in-process channel, TCP, or another adapter. The HTTP interface can read and replace these files through `/api/session/generated/requirements.lean` and `/api/session/generated/requirements.proto`, and `/api/session/generated/implementation.lean`. Validation, rendered views, metrics, reports, and OpenAPI descriptions are projections of the same model.

Session isolation is useful during an argument, but the current session files live below `/tmp/leanfm-sessions`. That is a workspace, not durable storage. A concluded argument must export or commit the accepted files—for example as `LeanFM/LLMGenerated/Requirements.lean`, `LeanFM/LLMGenerated/Requirements.proto`, and the separate `LeanFM/LLMGenerated/Implementation.lean`—together with any evidence needed by the project. Durability means that the specification has an ordinary name, history, review boundary, and reproducible checks independent of the chat service.

The server may use a language model to propose edits. Compilation and validation remain deterministic. The model is not the proof checker, and fluent prose is not evidence that an obligation holds.

6.3 What the Lean artifact must settle

The Lean file is the semantic source. It records actors, messages, per-task state machines, multi-party grammar productions, CSP-style processes, framing, properties, proof obligations, probability summaries, charts, and explanatory Markdown. A serious argument should force answers in each of the following dimensions.

Observable behavior

Which actor sends and receives each event? Which fields can an observer see? A message carries its `session` and `task`, its clock timestamp, and a list of prior event identifiers. Thus a join may cite several prior events, while several events may fork from the same prior event.

Component interactions

Global grammar productions state legal conversations; actor projections state what each component may send or receive. Composition must preserve endpoints, message types, correlation, and predecessor references. A diagram is useful evidence, but the grammar is the compositional object.

Statistical properties

User choices make the input probabilistic even when every server component is deterministic. Those choices propagate through the composed system as an MDP. The specification distinguishes unconstrained alternatives from weighted choices, states dwell-time distributions, and names reducers for success, latency, throughput, and queue occupancy. An observed event stream can then estimate the same quantities and be compared with the model.

Queueing and concurrency

Every actor has finite inbound and outbound queues. Several (`session`, `task`) pairs may be in flight at once, so a requirement must not silently reason about only one conversation. Full and empty queues produce observable blocking or sleeping behavior, and transport is a separate step from production and receipt.

Memory pressure

Message-count capacity alone is insufficient when payload sizes vary. A checkable requirement should declare a byte budget and derive queued bytes from the encoded envelopes. If active tasks retain additional state, the model must declare a bounded footprint for that state too. A representative obligation is

$$\text{AG}(\text{queuedBytes}(a) + \text{inFlightBytes}(a) \leq \text{budget}(a)).$$

This is a modeling obligation for LeanFM, not a claim that the present validator already proves heap usage of an arbitrary implementation.

6.4 Interrogate, characterize, synthesize, replay

The conversational interface is not merely a code generator. Its first job is to discover whether the desired claims are identifiable. It asks for scenario and task boundaries, start-to-end backpointers, concrete actor populations, clock and work units, observation windows, queue and memory limits, outage and recovery events, and the provenance of probabilities. A missing answer is reported as an indeterminate metric rather than replaced by an invented value.

Once those fields are available, one event stream should produce a standard characterization out of the box:

- one interaction diagram and derived state machine per scenario;
- XY line graphs for time-, load-, population-, and replica-indexed metrics;
- pie-chart histograms for categorical outcomes and distributions;
- uptime, outage fraction, reliability, and MTTR;
- client-experienced and server-aggregate throughput; and

- latency distributions and named percentiles.

Some streams contain observations; other requirements contain only expected distributions. These are labeled separately. An expected distribution can drive generation, but it is not presented as a measurement.

Generative similarity closes the loop. Reducers first turn the source stream into a named metric vector, including histogram bins as individual exact ratios. The generator produces a candidate scenario, serializes it to bytes, decodes it, and runs the same reducers. The candidate is accepted only when every required metric lies within its declared relative tolerance. Thus “similar” means measurement-equivalent for the stated characterization, not visually plausible or textually similar. Several very different event processes may satisfy the same finite metric vector; the grammar, invariants, and byte-level language still constrain which candidates are legal.

6.5 The protobuf artifact is part of the requirement

The parallel protobuf schema answers a question that a state diagram cannot: what bytes can cross a component boundary? Atomic payload messages and a `RequirementEnvelope.oneof` discriminator connect a grammar terminal to its wire representation. Validation checks, among other things, that field tags are nonzero and unique and that framing is explicit.

This pairing exposes useful disagreements. A Lean event may require a `priorIds` list while the wire schema provides only one predecessor. A latency property may depend on a clock that is never transmitted or recorded. A memory budget may be stated in bytes while an unbounded protobuf field remains. These are requirements defects, not serialization trivia.

The exact wire size is also an input to resource analysis:

```
queuedBytes actor observation :=
  (observation.inbound actor).sum encodedSize
  + (observation.outbound actor).sum encodedSize

memoryPressure actor observation :=
  queuedBytes actor observation
  + inFlightFootprint actor observation.activeWork
```

The first term can be computed from concrete envelopes. The second requires a declared abstraction of implementation state and must remain visibly labeled as such.

6.6 A ladder of checks

No single green check answers every kind of requirement. The workbench should present a ladder of evidence:

1. Lean elaboration and protobuf parsing establish well-formed artifacts.
2. Structural validation rejects missing actors, endpoints, terminals, framing, terminal states, process coverage, or real proof obligations.
3. Grammar and projected state-machine checks cover ordering and component interaction.

4. Temporal checking covers safety, reachability, and appropriately bounded liveness over observable states.
5. Probability and dwell reducers predict completion probability and latency; event-stream reducers estimate them from observations.
6. Queue and byte-budget checks cover capacity, backpressure, concurrency, and modeled memory pressure.
7. Rendered interaction diagrams, state machines, plots, and reports make the same typed model reviewable by people.

The distinction between model prediction and measurement must remain visible. An incomplete observation window may censor slow tasks; a frequency estimate is not automatically a proof of the declared probability; and a bounded trace does not establish unbounded liveness. Good generated reports say which claim was proved, calculated, sampled, or merely assumed.

6.7 How to argue well

The most useful prompts attack boundaries rather than ask for polish:

- “Show me the observable event that makes that sentence true.”
- “Which alternative ends here, and how is it bracketed in the grammar?”
- “What happens when this inbound or outbound queue is full?”
- “Run two sessions and two tasks concurrently; which events correlate?”
- “Which random choice belongs to the user, and how does it change the downstream completion and latency distributions?”
- “Derive the plot from the event stream, including incomplete tasks.”
- “Show the corresponding protobuf payload and its encoded-size bound.”
- “Is this result proved, model-checked, simulated, measured, or assumed?”

The argument is finished enough to review when deleting the transcript loses no normative information; a clean checkout can compile and validate the artifacts; every diagram and plot is derived rather than independently drawn; failures have observable outcomes; quantitative assumptions have units and scope; and every unsettled claim is an explicit obligation. That is the requirements-centered meaning of vibecoding in LeanFM: conversation discovers the specification, but the checked artifacts outlive the conversation.

Chapter 7

Worked Example: OAuth Authorization Code Flow

This chapter is a behavioural sketch inspired by the OAuth authorization-code flow. It is not a replacement for the OAuth specification and deliberately begins above HTTP and protobuf. Its purpose is to exercise scenario decomposition, parameter flow, visible evidence, alternatives, requirements, and rendering.

7.1 Actors, values, and observable events

The example has four participants: a person using a browser, an application, an authorization service, and a resource service. The browser is modelled separately from the person because redirects are observable messages between software participants.

```
inductive Actor where
  | user | browser | app | authorizationServer | resourceServer

opaque State      : Type
opaque AuthCode   : Type
opaque AccessToken : Type
opaque Resource   : Type
opaque Profile    : Type
opaque Password   : Type
opaque Bytes      : Type
```

Every terminal elaborates to the common event envelope. For example, the vague message “authorization redirected” becomes conceptually:

```
{ timeAt := t2, id := e2, priorIds := [e1]
, session := browserSession, task := oauthAuthorizationTask
, from := Actor.app, to := Actor.browser
, kind := "authorization redirected"
, fields := [field "state" state, field "redirectUri" callback]
}
```

Authentication detail used by Production 2

Before the redirect returns, the authorization service may authenticate the person. OAuth does not prescribe the authentication mechanism; we include a password branch to exercise knowledge flow without pretending that the password is an OAuth protocol parameter.

```
def authenticateAtAuthorizationServer (password : Password) : Scenario :=
  scenario "authenticate resource owner" do
    from_ .browser
    to_ .authorizationServer
    emitBytes "credentials submitted"
    (protectFor .authorizationServer
      (encode [field "password" password]))

    from_ .authorizationServer
    to_ .browser
    oneOf "authentication result"
    [ emitBytes "authentication accepted" (encode [tag 1])
      , emitBytes "authentication rejected" (encode [tag 0])
    ]
```

The wire bytes are specified even though the transport is not. The abstract `protectFor` constructor promises confidentiality to the named endpoint. A later refinement may realize that promise with TLS, message-level encryption, or a local trusted channel. Merely placing the password inside a field would not support the intended knowledge set: without protection, a network observer would also derive the password from the bytes.

7.2 Production 1: application request and redirect

```
def applicationRequestAndRedirect (resource : Resource) : Scenario :=
  scenario "application request and redirect" do
    let state <- declare State "state"
    let callback := Uri.https "app.example" "/oauth/callback"

    from_ .browser
    to_ .app
    emit "application request" [field "resource" resource]

    from_ .app
    to_ .browser
    emit "authorization redirected"
      [field "state" state, field "redirectUri" callback]

    export "state" state
    export "redirectUri" callback
    export "resource" resource
```

This production treats the application request and its redirect as one named unit. It establishes the correlation value `state` and exports it to the next production. Nothing here says how the application stores it.

7.3 Production 2: authorize, then grant or deny

```

def authorizationGranted : Scenario :=
  scenario "authorization granted" do
    use state <- imported State "state"
    let code <- declare AuthCode "code"

    from_ .authorizationServer
    to_ .browser
    emit "authorization code returned"
      [field "state" state, field "code" code]

    from_ .browser
    to_ .app
    emit "authorization callback"
      [field "state" state, field "code" code]
    export "code" code

def authorizationDenied : Scenario :=
  scenario "authorization denied" do
    use state <- imported State "state"
    from_ .authorizationServer
    to_ .browser
    emit "authorization error returned"
      [field "state" state, field "reason" "access_denied"]
    from_ .browser
    to_ .app
    emit "authorization error callback"
      [field "state" state, field "reason" "access_denied"]

def authorizeAndDecide (password : Password) : Scenario :=
  scenario "authorize and decide" do
    use state <- imported State "state"
    use redirectUri <- imported Uri "redirectUri"

    from_ .browser
    to_ .authorizationServer
    emit "authorization requested"
      [field "state" state, field "redirectUri" redirectUri]

    include (authenticateAtAuthorizationServer password)
    oneOf "authorization decision"
      [ include authorizationGranted
        , include authorizationDenied
      ]

```

The authorization request and its two possible continuations form one production. At a more abstract rendering level, the entire production can be shown as one node; expanding it reveals authentication and the explicitly bracketed choice. The abstract contract makes both decisions possible. A product model may later attach an estimated or measured probability distribution without changing the set of legal traces.

7.4 Scenario 3: exchange the code and use the token

```
def exchangeAndFetch : Scenario :=
  scenario "exchange code and fetch resource" do
    use code      <- imported AuthCode "code"
    use resource  <- imported Resource "resource"
    let token     <- declare AccessToken "accessToken"
    let profile   <- declare Profile "profile"

    from_ .app
    to_ .authorizationServer
    emit "token requested" [field "code" code]

    from_ .authorizationServer
    to_ .app
    emit "access token issued" [field "accessToken" token]

    from_ .app
    to_ .resourceServer
    emit "protected resource requested"
      [field "accessToken" token, field "resource" resource]

    from_ .resourceServer
    to_ .app
    emit "protected resource returned"
      [field "resource" resource, field "profile" profile]

    from_ .app
    to_ .browser
    emit "application response" [field "profile" profile]
```

The token itself is visible only at the endpoints where the abstract protocol says it is transmitted. The grammar makes no claim about the authorization server's database schema or the application's session representation.

7.5 Reading the scenarios together

```
def oauthAuthorization (resource : Resource) (password : Password) : Scenario :=
  scenario "OAuth authorization code flow" do
    include (applicationRequestAndRedirect resource)
    include (authorizeAndDecide password)
    after authorizationGranted include exchangeAndFetch

def oauthSystem (resource : Resource) (password : Password) : SystemSpec :=
  generatedBy (oauthAuthorization resource password)
  |> requireAll oauthRequirements
```

The two named productions provide the first abstraction level. A renderer may collapse either production to a single node or expand it to its messages and nested alternatives. The `after` line is a guarded nonterminal reference; the denial branch ends without exchanging a code.

7.6 The parallel protobuf alphabet

The scenario and its wire alphabet are maintained side by side. The scenario says which messages may occur, in what predecessor-order structure, and with which parameter relationships. The protobuf file assigns field numbers and concrete wire types to those same message atoms:

```
syntax = "proto3";

package leanfm.oauth;

enum Actor {
  ACTOR_UNSPECIFIED = 0;
  BROWSER = 1;
  APPLICATION = 2;
  AUTHORIZATION_SERVER = 3;
  RESOURCE_SERVER = 4;
}

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 clock_tick = 5;
  Actor src = 6;
  Actor dst = 7;

  oneof payload {
    ApplicationRequest application_request = 20;
    AuthorizationRedirect authorization_redirect = 21;
    AuthorizationRequest authorization_request = 22;
    CredentialsSubmitted credentials_submitted = 23;
    AuthenticationResult authentication_result = 24;
    AuthorizationCodeReturned authorization_code_returned = 25;
    AuthorizationCallback authorization_callback = 26;
    AuthorizationErrorReturned authorization_error_returned = 27;
    AuthorizationErrorCallback authorization_error_callback = 28;
    TokenRequest token_request = 29;
    AccessTokenIssued access_token_issued = 30;
    ProtectedResourceRequest protected_resource_request = 31;
    ProtectedResourceReturned protected_resource_returned = 32;
    ApplicationResponse application_response = 33;
  }
}

message ApplicationRequest {
  string resource = 1;
}

message AuthorizationRedirect {
  string state = 1;
  string redirect_uri = 2;
}

message AuthorizationRequest {
  string state = 1;
  string redirect_uri = 2;
}

message CredentialsSubmitted {
  bytes protected_credentials = 1;
}

message AuthenticationResult {
```

```

    bool accepted = 1;
}

message AuthorizationCodeReturned {
    string state = 1;
    bytes code = 2;
}

message AuthorizationCallback {
    string state = 1;
    bytes code = 2;
}

message AuthorizationErrorReturned {
    string state = 1;
    string reason = 2;
}

message AuthorizationErrorCallback {
    string state = 1;
    string reason = 2;
}

message TokenRequest {
    bytes code = 1;
}

message AccessTokenIssued {
    bytes access_token = 1;
}

message ProtectedResourceRequest {
    bytes access_token = 1;
    string resource = 2;
}

message ProtectedResourceReturned {
    string resource = 1;
    bytes profile = 2;
}

message ApplicationResponse {
    bytes profile = 1;
}

```

Each emitted scenario message maps to exactly one member of `EventEnvelope.payload`. The envelope carries the event identity, repeated `prior_ids`, (`session_id`, `task_id`) correlation key, monotonic clock tick, and endpoints outside the payload. The payload carries only the fields specific to that message.

Protobuf alone does not specify the protocol order, queue behavior, probability, or dwell time; those remain in the scenario. Conversely, the scenario does not invent byte tags. Validation checks that every scenario terminal has one protobuf variant, that its endpoints agree with the scenario, and that every referenced field has the expected protobuf type.

The schema determines valid protobuf encodings, but protobuf does not generally promise one canonical byte sequence. When byte-for-byte comparison is required, the refinement also fixes deterministic serialization and length-delimited message framing. An instantiated envelope can then be serialized and displayed as the actual bytes sent between the named actors.

7.7 Generate one legal trace and its artifacts

We can now run the specification in one direction. Grammar generation chooses productions and concrete field values; everything after that is a deterministic projection or reduction of the resulting events. For this run choose the authentication-accepted and authorization-granted alternatives and instantiate

```
session = "s1"           task = "oauth"
resource = "profile"     state = "x7"
redirectUri = "https://a/cb"
protectedCredentials = aa bb
code = 01 02             accessToken = 03 04
profile = 05 06
```

The production derivation is

```
oauth
=> applicationRequestAndRedirect ;
   authorizeAndDecide ;
   after(authorizationGranted, exchangeAndFetch)
=> applicationRequestAndRedirect ;
   authorizationRequested ;
   credentialsSubmitted ; authenticationAccepted ;
   authorizationGranted ; exchangeAndFetch
```

Because `authorizationGranted` was selected, the `after` guard admits `exchangeAndFetch`. Selecting the denial alternative would disable that production. Expanding every remaining nonterminal yields this terminal word:

```
e1: Browser → Application: applicationRequest
e2: Application → Browser: authorizationRedirect
e3: Browser → Authorization: authorizationRequest
e4: Browser → Authorization: credentialsSubmitted
e5: Authorization → Browser: authenticationResult(true)
e6: Authorization → Browser: authorizationCodeReturned
e7: Browser → Application: authorizationCallback
e8: Application → Authorization: tokenRequest
e9: Authorization → Application: accessTokenIssued
e10: Application → Resource: protectedResourceRequest
e11: Resource → Application: protectedResourceReturned
e12: Application → Browser: applicationResponse
```

The choice has now disappeared into ordinary terminals. Event e5's payload bytes select the accepted authentication result. Event e6's authorizationCodeReturned oneof tag, source, and bytes distinguish the grant branch from the error-returned branch. No independent “chooser” value is needed.

7.7.1 Envelope stream and concrete bytes

Assign monotonically increasing clock ticks and event identifiers. This example has a linear predecessor chain, so $\text{priorIds}(e_1)$ is empty and $\text{priorIds}(e_n)$ is $[e_n - 1]$. A fork or join would make these lists non-linear without changing the procedure. Deterministic protobuf serialization of the schema on the preceding pages produces:

```
# Deterministic protobuf encodings of one legal OAuth trace.
# Columns: event-id, clock, payload alternative, encoded length, envelope bytes.
e1 0 application_request      31 0a0265311a02733122056f6175746830013802a201090a0770726f666696c65
e2 2 authorization_redirect   46 0a026532120265311a02733122056f61757468280230023801aa01120a027837120c68747470733a2f2f612f6362
e3 4 authorization_request    46 0a026533120265321a02733122056f61757468280430013803b201120a027837120c68747470733a2f2f612f6362
e4 5 credentials_submitted    32 0a026534120265331a02733122056f61757468280530013803ba01040a02aabb
e5 7 authentication_result     30 0a026535120265341a02733122056f61757468280730033801c201020801
e6 9 authorization_code_returned 36 0a026536120265351a02733122056f61757468280930033801ca01080a02783712020102
e7 11 authorization_callback   36 0a026537120265361a02733122056f61757468280b30013802d201080a02783712020102
e8 14 token_request            32 0a026538120265371a02733122056f61757468280e30023803ea01040a020102
e9 16 access_token_issued      32 0a026539120265381a02733122056f61757468281030033802f201040a020304
e10 18 protected_resource_request 42 0a03653130120265391a02733122056f61757468281230023804fa010d0a020304120770726f666696c65
e11 21 protected_resource_returned 43 0a0365313112036531301a02733122056f6175746828153004380282020d0a0770726f666696c6512020506
e12 23 application_response    34 0a0365313212036531311a02733122056f617574682817300238018a02040a020506
```

These are complete EventEnvelope encodings, not illustrative message tags. They were generated with deterministic protobuf serialization. The twelve envelopes occupy 440 bytes; the largest is 46 bytes. Protobuf deterministic mode makes this fixture reproducible for a given implementation, while the grammar's semantic requirement remains successful decoding to the same terminals and field values.

7.7.2 Residual state-machine path

Parsing each envelope selects a payload alternative, checks its declared src/dst, and consumes one grammar terminal. The residual after each prefix is therefore:

```
oauth
--e1--> waitingForRedirect
--e2--> waitingForAuthorizationRequest
--e3--> authenticating
--e4--> waitingForAuthenticationResult
--e5--> waitingForAuthorizationDecision
--e6--> waitingForCallback
--e7--> waitingForTokenRequest
--e8--> waitingForToken
--e9--> waitingForResourceRequest
--e10--> waitingForResource
--e11--> waitingForApplicationResponse
--e12--> done
```

This path is one highlighted route through the generated state machine. The machine itself is derived from all grammar residuals and therefore also contains the rejected-authentication and denied-authorization paths.

7.7.3 Actor projections

Projection erases terminals an actor neither sends nor receives while preserving the order of the rest. The same word generates four local replay scripts:

Actor	Projected actions
Browser	send e1; receive e2; send e3; send e4; receive e5; receive e6; send e7; receive e12
Application	receive e1; send e2; receive e7; send e8; receive e9; send e10; receive e11; send e12
Authorization Resource	receive e3; receive e4; send e5; send e6; receive e8; send e9 receive e10; send e11

Each line can become a component conformance test. The global word remains the authority for cross-component order and parameter equality; a local projection does not grant an actor knowledge of erased events.

7.7.4 Event order, queues, and memory

The `priorIds` fields generate a twelve-node order graph with edges $e_i \rightarrow e_{i+1}$. The interaction diagram is the same graph laid out by actor; the state-machine path lays it out by grammar residual. Neither is independently authored.

The message word does not uniquely determine a transport schedule. For a concrete queue artifact, choose the canonical schedule that performs, for each event, `produce`; `transport`; `receive` before generating the next event. With positive inbound and outbound capacities, each queue peaks at one message. The queued-byte peak for an actor is the largest encoded envelope waiting at that actor in this schedule, and the global queued-byte peak is 46. A different legal interleaving may have a larger peak and must be explored when checking the universal memory-budget property. The selected schedule is a witness, not the capacity proof.

7.7.5 Measurements, plots, and verdicts

From the clocks and correlation key, the generated task attempt is

```
{ key = ("s1", "oauth"),
  started = { kind = applicationRequest, timeAt = 0 },
  completed = { kind = applicationResponse, timeAt = 23 },
  outcome = succeeded, latency = 23 ticks }
```

It contributes one point (`maxConcurrency=1`, `latency=23`) to the latency XY plot, a cumulative-byte series ending at (`clock=23`, `bytes=440`), and one successful slice to the outcome chart. It does *not* establish a 100 percent success probability. The grammar-level weight predicts 0.87 mass on the grant branch; estimating that probability requires many generated or observed sessions, including incomplete ones.

Trace-local monitors accept the matching `state=x7`, the code flow 01 02, and the fact that no exchange follows denial. CTL obligations such as “every decision reaches the application” are not proved by this trace; the checker must examine every reachable grammar residual. The complete artifact pipeline is thus

```

grammar + field values
-> legal terminal word
-> typed EventEnvelope stream
-> framed protobuf bytes
-> partial-order graph + interaction diagram
-> residual FSM path + per-actor projections
-> queue schedule + byte-pressure samples
-> task attempt + probability/latency/XY reducers
-> replay fixture + trace-local property verdicts

```

Every arrow is reproducible. Only the initial branch and value generation, and the separately stated transport schedule, add choices.

7.8 Interaction diagrams by scenario

7.9 Requirements as observable constraints

```

def oauthRequirements : List Requirement :=
  [ require "callback state matches request"
    (on "authorization callback" (previouslySame "state"
      "authorization requested"))

    , forbid "code exchange after denial"
    (previously "authorization denied" && event "token requested")

    , require "issued token came from a code"
    (on "access token issued" (previouslySame "code" "token requested"))

    , ctl "every decision reaches the application"
    (AG (decisionAtAuthorizationServer --> AF callbackAtApplication))

    , ctl "a protected result is never returned without a token request"
    (AG (resourceReturned --> previously tokenRequested))
  ]

```

7.9.1 Who knows the password?

Observable bytes are not the same as knowledge of their plaintext. Rather than introducing “secrecy” as a special property, calculate the actors that can derive the password from initial knowledge and the event list.

```

def passwordKnowers (trace : Trace WireEvent) : List Actor :=
  allActors.filter fun actor =>
    Derives (knowledgeFrom actor trace) (value password)

theorem oauth_password_knowers
  (trace : Trace WireEvent)
  (accepted : (oauthSystem resource password).Accepts trace)

```

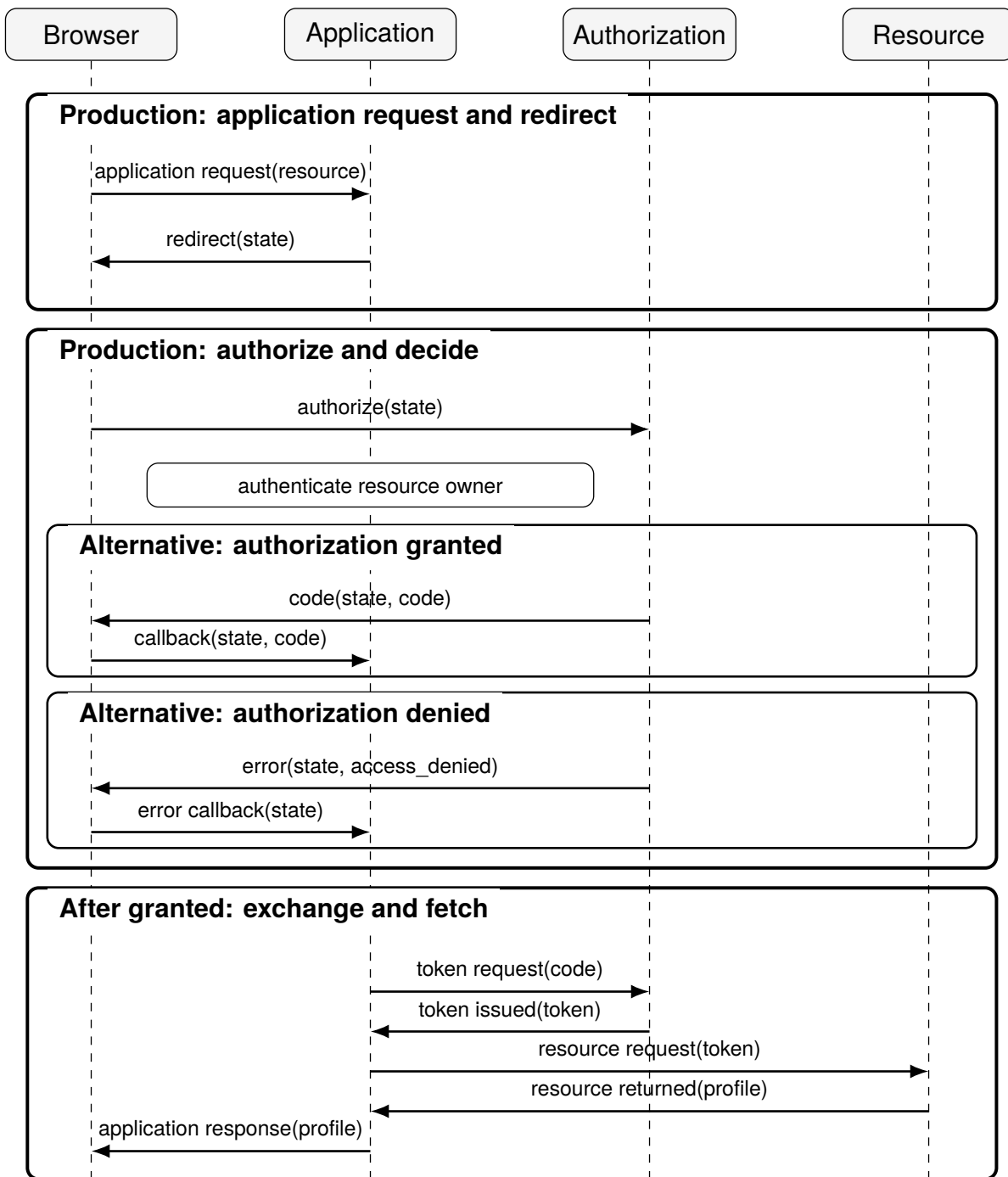


Figure 7.1: OAuth interaction view. Production frames provide collapsible abstraction boundaries. The authorization production contains its request and both alternatives; the guarded exchange production is enabled only by the granted alternative.

```
(initial : InitiallyKnown password [.browser, .authorizationServer])
(honest : Uncompromised [.browser, .authorizationServer])
(protection : PerfectProtection protectFor) :
  passwordKnowers trace = [.browser, .authorizationServer] := by
  exact knowledgeClosureOfProtectedField accepted initial honest protection
```

The result says directly who knows the value. A password typed into the modeled browser is derivable by that browser; hiding this behind the word “secret” would lose information. If a narrower result is required, the actor model must represent a trusted credential component separately. The calculation remains conditional: a compromised endpoint, logging event, newly visible key, or failed protection refinement changes the knowledge closure and therefore changes the returned set.

7.9.2 Byte-level refinement without choosing a transport

Every vague OAuth message can be assigned a canonical byte term before HTTP or protobuf is chosen:

```
refineMessage "authorization requested" where
  term := tuple
    [ tagBytes 0x01
      , namedBytes "state" (encode state)
      , namedBytes "redirectUri" (encode callback)
    ]
  bytes := encodeCanonical term

refineMessage "token requested" where
  term := tuple [tagBytes 0x02, namedBytes "code" (encode code)]
  bytes := encodeCanonical term
```

This fixes framing, tags, and value relationships at an abstract byte-algebra level. A subsequent HTTP mapping might put those bytes into form fields; a protobuf mapping might use numbered fields. Either concrete alphabet must prove that decoding and re-encoding preserve the abstract terminal and its parameter bindings.

The use of previously is deliberately a trace predicate in this first version. It can later receive dedicated past-time syntax without changing the event model.

7.10 Probabilities and charts

The authorization service need not be random. The user supplies random input by granting or denying consent; the browser, queues, transport, and authorization service then react deterministically to the input they receive:

```
def consentDecision : Scenario :=
  weightedOneOf "consent decision"
    [ weighted (87 / 100) authorizationGranted
      , weighted (13 / 100) authorizationDenied ]

def oauthCharts : List ChartSpec :=
  [ pie "authorization outcomes" do
    source events; groupBy terminalOutcome; value count
```

```

, line "end-to-end authorization latency" do
  source (correlate events by "state")
  x clock; y (elapsed "authorization requested" "application response")
]

```

No separate field says that the user “owns” this choice. The grant and denial branches resolve to different consent-event bytes, and that terminal’s source identifies the user-facing actor that emitted them. After those bytes select a branch, every transition may have probability one and the global model is still probabilistic. The grant branch carries probability mass 0.87 through the outbound queue, transport, the server’s inbound queue, and the deterministic grant response. The denial branch similarly carries mass 0.13. Deterministic processing transports probability mass; it does not remove it.

For the simplified path with no loss or timeout,

$$P(\text{grant callback}) = 0.87 \cdot 1 \cdot 1 \cdot 1 = 0.87, \quad P(\text{denial callback}) = 0.13.$$

The factors of one are deterministic enqueue, transport, and server steps. Their dwell times still affect the latency distribution even though they do not split the probability mass.

With queue scheduling and other unresolved choices included, the complete system is therefore a Markov decision process. Probabilistic edges describe inputs or measured outcomes; nondeterministic edges describe decisions not assigned a distribution, such as which enabled actor is scheduled next. An analysis first fixes or quantifies over a scheduling policy and then propagates the user-originated mass through the resulting paths.

For several sessions in flight, the MDP state contains every actor’s bounded queues and its per-(session, task) residuals. One user’s random event can change later queue pressure and scheduling possibilities for other sessions, so the probabilistic state is the composed system state rather than a special marker attached only to the user actor.

The percentages remain model inputs or measurements, not truths inferred from the grammar. The grammar defines which behaviours are possible; the probabilistic refinement says how mass is distributed over them.

7.11 Residual state machine

This automaton is intentionally behavioural. It does not contain browser cookies, database rows, server threads, or cryptographic library state. Those details may be essential to an implementation proof, but they are refinements of this contract, not the contract itself.

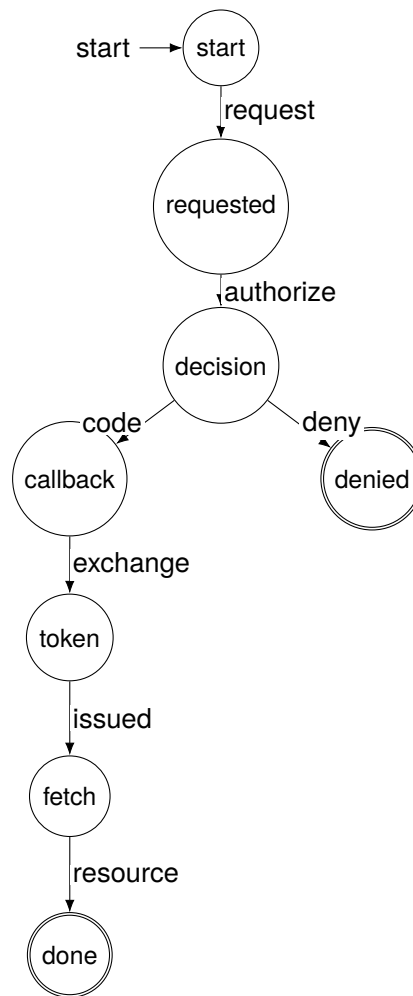


Figure 7.2: OAuth state machine derived from legal grammar continuations.

Chapter 8

CTL over Generated Futures

A grammar generates a branching transition system. CTL speaks directly about that branching, but its vocabulary becomes clearer when placed between basic modal logic and linear temporal logic. Path quantifiers distinguish *there exists a future* from *every future*, while temporal operators distinguish the next state, eventuality, invariance, and until.

The author’s modal-logic background used here was learned mostly from Kr"oger and Merz [5]. This citation identifies the source of that education; it is not a claim that the authors originated modal or temporal logic.

8.1 Basic modal logic

Propositional logic evaluates a formula at one world or state. Basic modal logic adds an accessibility relation R between worlds and two operators:

$$\begin{aligned} w \models \Box p & \text{ iff } p \text{ holds at every } v \text{ such that } wRv, \\ w \models \Diamond p & \text{ iff } p \text{ holds at some } v \text{ such that } wRv. \end{aligned}$$

The readings are “necessarily p ” and “possibly p .” Under classical negation they are dual: $\Diamond p \equiv \neg \Box \neg p$. The symbols do not fix the meaning of accessibility. Depending on the model, an accessible world may be a next computation state, a world compatible with an actor’s knowledge, a normatively permitted world, or something else.

The basic normal modal logic K allows arbitrary Kripke frames. Additional frame conditions validate additional principles: reflexivity, transitivity, symmetry, and the Euclidean condition lead to familiar stronger systems. Those principles must not be imported merely because the English word “necessarily” sounds strong. LeanFM uses a directed computation-successor relation and does not silently treat it as reflexive, symmetric, or epistemic.

8.2 LTL: time along one path

Linear Temporal Logic (LTL) describes one linear path with operators including X (next), F (eventually), G (always), and U (until). Thus Fp says that p occurs at some suffix of the selected path, while Gp says that p holds at every suffix of that path.

LTL's temporal words answer *where along this path*; they do not themselves choose between one possible branch and every branch of a transition system. A model-checking convention often asks whether every initial path satisfies an LTL formula. That outer convention should not be confused with a path quantifier in the LTL syntax. For example, asking whether some legal execution eventually reaches recovery and asking whether all legal executions eventually reach recovery require different branching claims even though both contain the words “eventually recovery.”

LeanFM does not currently implement LTL. It uses CTL because the requirements deliberately combine two independent axes:

- *always* or *eventually* says where along forward time a predicate must hold; and
- *necessarily* or *possibly* says whether it must hold on every permitted forward continuation or may hold on at least one.

8.3 CTL: path modality combined with time

CTL pairs a path quantifier with each temporal operator. E is existential: there is at least one permitted continuation. A is universal: every permitted continuation satisfies the temporal condition. In the vocabulary of modal logic, E supplies temporal possibility and A supplies temporal necessity.

“Necessary” and “possible” are therefore future-relative readings here. They quantify over computations from the current observation, not over timeless logical worlds. The four most important combinations can be read literally:

$EF p$: possibly eventually p ,	$AF p$: necessarily eventually p ,
$EG p$: possibly always p ,	$AG p$: necessarily always p .

The temporal word and modal word do different work. F versus G chooses eventuality versus invariance *along* a continuation. E versus A chooses some versus all permitted continuations. Neither axis should be omitted when stating a branching-time requirement.

An observation is immutable once recorded. A value changes only by moving forward to a successor observation; it is never updated retroactively inside the current state. This does not mean that values are monotone—a queue may grow and later shrink—but every change occurs on a forward transition.

Formula	Reading	Typical use
$EF p$	possibly, eventually p	temporal possibility / witness
$AF p$	necessarily, eventually p	termination / response
$EG p$	possibly, always p	viable invariant path
$AG p$	necessarily, always p	safety invariant
$EX p$	possibly, next p	enabled successor / one-step witness
$AX p$	necessarily, next p	required one-step consequence
$(p AU q)$	every path keeps p until q	constrained progress
$(p EU q)$	some path keeps p until q	possible progress
$(p AW q)$	every path: p until q , or p forever	necessary weak until
$(p EW q)$	some path: p until q , or p forever	possible weak until

For example, $AF p$ is not merely “eventually p ” and $EF p$ is not a weaker spelling of the same linear claim. The first requires p on every legal future generated from the current trace prefix; the second needs one legal future as a witness. Branching is part of the requirement.

Strict CTL requires every temporal operator to be governed by A or E . CTL* is a more general branching-time language in which state formulas and path formulas can be nested more freely; LeanFM does not implement CTL*. This matters when translating prose: not every arbitrary alternation of “on all paths” and “eventually” is a formula of the small CTL datatype used here.

8.4 General and multiple modalities

General modal logic need not have only one accessibility relation. A multimodal model can use $\Box_t/\Diamond_t$ for temporal reachability, $K_a/\widehat{K}_a$ for what actor a knows or considers possible, and a different pair for permission or obligation. Dynamic logic can index modalities by programs. Tense logics can add past-directed relations. These systems share the Kripke-style idea of evaluating modal formulas through accessibility, but their modalities are not interchangeable.

LeanFM’s CTL A and E range only over legal forward computations generated by the current model. They do not mean:

- probability one versus positive probability;
- known versus merely believed by an actor;
- obligatory versus permitted by a policy; or
- physically necessary versus logically possible.

Probability, knowledge, authorization, and normative requirements need their own named structures or modalities. Keeping the relations separate prevents an easy category error: a branch can be computationally possible while having unknown probability, being unknown to an actor, or being forbidden by policy.

8.5 Possibility means preparedness

There is a practical consequence to treating possibility as a first-class temporal mode. When a requirement declares $EF p$, it does more than acknowledge that a model checker can find an accidental path to p . It says that p is a supported future for which the system has a plan.

A possibly obligation therefore contains three linked artifacts:

1. a legal grammar branch and witness trace reaching the condition;
2. a handling plan in `Requirements.lean`; and
3. generated implementation code whose justification cites that possibility requirement.

For example, “a bakery delivery may fail” requires a decodable `DeliveryFailed` message, a legal failure branch, and code that performs the specified refund or notification behavior. Removing every failure branch does not implement reliability; it makes the real failure trace out-of-spec.

Possibility is distinct from probability. A possible condition may have a small estimated probability, an expected distribution, or no known probability at all. Conversely, assigning positive probability to a condition makes its branch possible. The handling obligation comes from supported possibility, not from whether the estimated percentage crosses a threshold.

8.6 Strong implication and the Chewbacca argument

The name “Chewbacca defense” is a cultural reference to the *South Park* episode “Chef Aid,” whose courtroom parody uses an irrelevant claim about Chewbacca to distract from the conclusion at issue. The reference supplies a memorable example of non sequitur; it is not a technical term originating in modal or temporal logic.

Material implication is weak in the precise sense that $p \rightarrow q$ is true whenever p is false. That is useful inside a state, but it permits a vacuous global claim when no reachable state ever satisfies the antecedent. LeanFM names the non-vacuous temporal combination

$$p \Rightarrow_s q \stackrel{\text{def}}{=} EF p \wedge AG(p \rightarrow q).$$

The first conjunct requires a legal future witnessing p . The second requires that, necessarily always, every reachable occurrence of p also satisfies q . Together they imply $EF q$: the p witness cannot avoid q .

The deliberately absurd test uses

$$\begin{aligned} p &= \text{“Chewbacca lives on Endor”}, \\ q &= \text{“you must acquit”}. \end{aligned}$$

A Chewbacca argument cannot obtain q merely by mentioning p . To enter the requirements as strong implication, the model must admit a reachable p witness and establish $AG(p \rightarrow q)$ over every permitted continuation. If the observable grammar supplies no relation between residence and acquittal, that second obligation has no justification and is rejected. Logic cannot determine the meaning or relevance of arbitrary English propositions by itself; the useful point is that neither a missing antecedent nor a single rhetorical occurrence of it can make the formal implication pass vacuously.

In Lean the definition is direct:

```
def stronglyImplies (p q : CTL S) : CTL S :=
  CTL.and (CTL.ef p) (CTL.ag (CTL.implies p q))
```

The path quantifier is part of the until operator. We write AU and EU as binary algebraic operators, not an outer A or E wrapped around a bracketed until formula. Consequently a subexpression never changes meaning merely because another expression surrounds it:

```
def boundedUntilTerminal : CTL Observation :=
  atom withinCapacity AU atom isTerminal

def someAuthorizedContinuation : CTL Observation :=
  atom hasValidAuth EU atom isSucceeded
```

8.7 Weak until: AW and EW

Strong until requires the right operand eventually to occur. Weak until permits it never to occur, provided the left operand continues to hold. Thus

$p \text{ AW } q$: on every future, p holds until q , or p remains forever,
 $p \text{ EW } q$: on some future, p holds until q , or p remains forever.

The boundary state satisfying q need not also satisfy p ; p is required only before q . On a future where q never occurs, every state must satisfy p . This makes weak until useful for requirements such as “necessarily remain available until shutdown, while shutdown is permitted but not required.”

For existential paths, $p \text{ EW } q$ is equivalent to $(p \text{ EU } q) \vee \text{EG } p$. The analogous-looking $(p \text{ AU } q) \vee \text{AG } p$ is *not* a correct definition of AW on a branching system. Some futures may reach q while other futures preserve p forever; neither global disjunct need hold even though every individual future satisfies weak until. LeanFM therefore implements AW directly.

```
def serviceMayRunUntilShutdown : CTL Observation :=
  atom available AW atom shutdown

def someWorkerCanRemainReady : CTL Observation :=
  atom ready EW atom retired
```

This convention also makes tree structure explicit. In $r \ \&\& \ (p \text{ AU } q)$, the right operand is the complete AU expression; there is no context-sensitive reinterpretation of $[p \text{ U } q]$.

For a request event carrying identifier r , an expected response property might say that on every continuation a matching success or failure eventually occurs. Authentication might say that every successful use of a session has an earlier issuance event with the same session identifier.

8.8 Past claims

CTL is future-facing at the current state, but security and protocol claims often refer to evidence in the past: “this capability was previously issued” or “this response matches an earlier request.” Since states are trace prefixes, past facts can initially be expressed as predicates over the accumulated trace. This gives a clear baseline without prematurely committing the surface language to a particular past-time temporal logic.

There is still an open design question. Explicit past operators may make properties more readable and allow better checking algorithms. The project should compare trace predicates, CTL with past operators, and compilation of finite-history monitors into state. The criterion is maintainability of specifications as the event schema evolves.

8.9 Fairness and time bounds

AF does not by itself distinguish a participant that is perpetually denied a turn from one that violates the protocol. Fairness assumptions must therefore be named, scoped, and visible in theorem statements. Similarly, “eventually” is not “within 200 milliseconds.” Quantitative bounds belong in an explicit timed layer rather than being smuggled into timestamps and informal prose.

Chapter 9

Lean as the Rendered Language

The user-facing notation should render to a conservative subset of Lean. There is no second opaque specification file: domain declarations are Lean definitions, expected behaviours are Lean values, and requirements are Lean propositions or checked queries.

```
def checkout : Grammar CheckoutEvent :=
  request >>> (accepted <||> rejected)

def eventuallyAnswered : CTL Observation :=
  CTL.ag (CTL.implies requestPending
    (CTL.af (CTL.or requestAccepted requestRejected)))
```

Most semantic machinery should be imported from a fixed library. A system author normally supplies only:

- typed actors, identifiers, event kinds, and payloads;
- parameterised grammar definitions;
- projections from trace prefixes to observable propositions;
- named temporal requirements and proofs or decidable checks.

This boundary matters for automation. Generated Lean remains small enough to review, while library proofs and algorithms are reused rather than regenerated.

9.1 From specification to service

Lean can host executable code, including a small web server. That makes the formal artifact part of the running project rather than a document beside it. The same typed definitions can drive validation endpoints, test-trace generation, protocol schemas, visualisations, and runtime monitors. Extraction and integration choices must preserve the distinction between a verified behavioural boundary and an unverified infrastructure layer.

Chapter 10

Research and Implementation Roadmap

The project will proceed through small executable examples rather than attempting a complete logic and compiler at once.

10.1 Milestones

1. Stabilise the event envelope, including identity, predecessor links, actor endpoints, kind, payload, and a disciplined account of time.
2. Give the grammar algebra a trace semantics and establish laws for sequence, choice, guarded context, and interleaving.
3. Define parameter binding and participant projection, then test both on authentication and ordinary web request protocols.
4. Preserve named scenario boundaries while reading their references into one grammar; render interaction diagrams and residual state machines per scenario.
5. Treat independent requirements as trace-language intersections and report minimal inconsistent requirement sets.
6. Generate finite transition systems and evaluate the CTL fragment already represented by the LeanFM library.
7. Compare past-trace predicates with explicit past operators.
8. Render declarative pie and line charts as reductions over observable events.
9. Connect specifications to a Lean web service and generated protocol schemas, preserving one source of typed domain truth.

10.2 Proof obligations

The initial metatheory should make the following questions precise:

- Does a projected local behaviour faithfully implement its part of the global grammar?

- Under what finiteness restrictions is CTL checking decidable and executable?
- Which guards can be compiled into finite monitor state?
- When does modular composition preserve an established temporal property?
- Which fairness and timing assumptions are required by a liveness theorem?

The intended outcome is not merely a verifier. It is a development loop in which writing the behaviour is the fastest way to obtain working, reviewable software.

Part II

Security Protocol Exercises from the CSP Approach

Chapter 11

From CSP Roles to Multiparty Grammars

This part revisits examples from Ryan et al. [10] and its official Casper companion scripts. The goal is not to transliterate CSP operators. It is to use well-known security protocols as demanding exercises for the grammar-based method: multiple roles, fresh values, encrypted components, an active intruder, calculated knowledge sets, agreement, and messages whose legality depends on the preceding trace.

11.1 Can actors really be specified separately?

Yes, provided composition is defined as synchronization over the same observable events rather than as concatenation of independent scripts. Each actor accepts a language over its own projection of the global event alphabet.

```
structure RoleSpec where
  actor      : Actor
  accepts    : Grammar (ProjectedEvent actor)
  initialKnowledge : Knowledge actor

def composeRoles (roles : List RoleSpec) : GlobalGrammar :=
  synchronize roles where
    sendReceiveAgree := sameEventId && sameBytes && oppositeEndpoints
```

For a global trace t , let $t \upharpoonright_a$ retain events sent or received by actor a . The composition accepts exactly when every local role accepts its view:

$$L(\text{composeRoles}(R_1, \dots, R_n)) = \{t \mid \forall i, t \upharpoonright_{a_i} \in L(R_i)\}.$$

One global message is therefore not duplicated into unrelated send and receive events. It has a single identity, byte string, sender, and recipient; both endpoint grammars constrain it. An unmatched send, an unexpected receive, or disagreement about bytes makes the intersection empty at that trace prefix.

11.1.1 Local state without global implementation state

A role may bind parameters from the events it observes. Those bindings are a parser environment for the trace, not a claim about implementation memory. A role can say “the nonce in this reply equals

the nonce I previously sent” without specifying a table, variable layout, or retry mechanism.

11.2 A transport-neutral byte algebra

Security claims need more than message names. They need an algebra that says which bytes expose which values. We still need not choose HTTP, protobuf, ASN.1, or a particular cipher suite.

```
abbrev Bytes := ByteArray

inductive WireTerm where
| atom      : Name -> Bytes -> WireTerm
| tuple     : List WireTerm -> WireTerm
| encrypt   : KeyRef -> WireTerm -> WireTerm
| sign      : KeyRef -> WireTerm -> WireTerm

structure WireEvent where
  timeAt : ClockTimestamp
  id      : EventId
  priorIds : List EventId
  session : SessionId
  task    : TaskId
  from    : Actor
  to      : Actor
  term    : WireTerm
  bytes   : Bytes
  encodedBy : bytes = encodeCanonical term
```

At this level, encryption is a symbolic constructor with an encoding into bytes. The intruder may concatenate, split visible tuples, encrypt with known keys, and decrypt only with a derivable inverse key. A concrete refinement must replace this idealized algebra with a transport and cryptographic implementation whose security assumptions justify the abstraction.

11.3 Yahalom as three separate actors

Yahalom is an excellent first exercise because an initiator, responder, and trusted server exchange fresh nonces and receive a session key. The companion model states secrecy for the session key and agreement claims involving the nonces and key.

Let K_A and K_B be long-term keys shared by the server with Alice and Bob, N_A and N_B fresh nonces, and K_{AB} the proposed session key. The following actor grammars use symbolic wire terms whose canonical encodings are byte strings.

11.3.1 Alice, the initiator

```
def aliceRole (alice bob server : Actor) (ka : SharedKey) : RoleSpec :=
  role alice "Yahalom initiator" do
    let na <- fresh Nonce "na"

    from_ alice; to_ bob
```

```

send (atom "initiator nonce" na.bytes)

from_ server; to_ alice
receive (encrypt ka.ref
  (tuple [actor bob, sessionKey "kab", nonce na, nonce "nb"]))
  where_ field "na" == na
let kab <- take SessionKey "kab"
let nb  <- take Nonce "nb"
let ticketForBob <- take WireTerm "ticketForBob"

from_ alice; to_ bob
send ticketForBob
send (encrypt kab.ref (nonce nb))

```

Alice's `where_` clause is the context-sensitive step: the received nonce must equal the value generated earlier. The surface structure is context-free, but legality depends on the binding environment accumulated from the prefix.

11.3.2 Bob, the responder

```

def bobRole (alice bob server : Actor) (kb : SharedKey) : RoleSpec :=
  role bob "Yahalom responder" do
    from_ alice; to_ bob
    receive (atom "initiator nonce" ?naBytes)
    let na <- take Nonce "na"
    let nb <- fresh Nonce "nb"

    from_ bob; to_ server
    send (encrypt kb.ref
      (tuple [actor alice, nonce na, nonce nb]))

    from_ alice; to_ bob
    receive (encrypt kb.ref (tuple [actor alice, sessionKey "kab"]))
    let kab <- take SessionKey "kab"
    receive (encrypt kab.ref (nonce nb)) where_ field "nb" == nb

```

Bob need not know Alice's local control state. He constrains only the events visible at his boundary and the values learned from them.

11.3.3 The trusted server

```

def serverRole (alice bob server : Actor)
  (ka kb : SharedKey) : RoleSpec :=
  role server "Yahalom key server" do
    from_ bob; to_ server
    receive (encrypt kb.ref
      (tuple [actor alice, nonce "na", nonce "nb"]))
    let na <- take Nonce "na"
    let nb <- take Nonce "nb"
    let kab <- fresh SessionKey "kab"

```

```

let bobTicket := encrypt kb.ref
  (tuple [actor alice, sessionKey kab])
from_ server; to_ alice
sendMany
  [ encrypt ka.ref
    (tuple [actor bob, sessionKey kab, nonce na, nonce nb])
  , bobTicket
  ]

```

11.3.4 Global assembly and context conditions

```

def yahalom : GlobalGrammar :=
  composeRoles
    [ aliceRole .alice .bob .server keyAlice
      , bobRole .alice .bob .server keyBob
      , serverRole .alice .bob .server keyAlice keyBob
      , intruderRole .ivo
    ]
  |> where_ (distinct [.alice, .bob, .server, .ivo])
  |> where_ (freshValuesNeverRepeat ["na", "nb", "kab"])
  |> where_ (longTermKeysKnownOnlyToOwnersAndServer)

```

The three honest roles can be authored and checked separately. Composition exposes incompatibilities: for example, if Alice expects the server's first encrypted tuple to name Bob but the server grammar could name another responder, the shared global event cannot satisfy both projections.

11.4 The attacker is another grammar plus deduction

An active network attacker observes public wire events, blocks them, replays known bytes, and emits any term derivable from current knowledge. It does not learn the plaintext of an encryption merely because its encoded bytes are observable.

```

inductive Derives : Knowledge -> WireTerm -> Prop where
| initial : term in knowledge -> Derives knowledge term
| tuple   : (forall x in xs, Derives knowledge x) ->
  Derives knowledge (.tuple xs)
| project : Derives knowledge (.tuple xs) -> x in xs ->
  Derives knowledge x
| encrypt : Derives knowledge key -> Derives knowledge body ->
  Derives knowledge (.encrypt key body)
| decrypt : Derives knowledge (.encrypt key body) ->
  Derives knowledge (inverse key) -> Derives knowledge body

def intruderRole (ivo : Actor) : RoleSpec :=
  adversary ivo do
    observe publicWireEvents
    mayBlock
    mayReplay

```

```
maySend fun knowledge term => Derives knowledge term
```

This is the grammar analogue of the CSP/Dolev–Yao attacker: possible hostile messages are generated from observable history and constrained by a context-sensitive derivability relation.

11.5 Knowledge and agreement become theorems

Knowledge is calculated for every actor from every accepted trace, not merely the happy-path scenario. A traditional secrecy claim is recovered by comparing that set with the actors permitted to know the value. For Yahalom:

```
theorem yahalom_session_key_knowers
  (trace : Trace WireEvent)
  (accepted : yahalom.Accepts trace)
  (honest : Uncompromised [.alice, .bob, .server]) :
  knowers (sessionKey accepted.kab) trace = [.alice, .bob, .server] := by
  -- proof uses the fixed symbolic-crypto library
  exact knowledgeClosureFromKeyOwnership accepted honest

theorem yahalom_bob_agrees_with_alice
  (trace : Trace WireEvent)
  (accepted : yahalom.Accepts trace)
  (bobDone : completed trace .bob) :
  exists run, peer run == .alice &&
    run.na == bobDone.na && run.nb == bobDone.nb := by
  exact agreementFromNonceChallenge accepted bobDone
```

The exact proof scripts depend on the library still to be built. The important design constraint is already visible: theorem statements refer only to accepted event traces, knowledge derived from those traces, and explicit compromise assumptions.

11.6 Otway–Rees: guards are not optional decoration

The Otway–Rees companion model introduces a run marker M and guards that the two participants differ. Each participant encrypts its nonce, marker, and actor names under its long-term server key. The server returns separate encrypted components.

This example adds three useful `where_` obligations:

```
where_ alice != bob
where_ response.field "marker" == request.field "marker"
where_ encryptedPart.names == [alice, bob]
```

The grammar shape remains a small sequence of messages. The accepted language is not context-free once equality of an unbounded marker and actor parameters matters. Keeping these tests as named guards preserves the readable protocol skeleton while making the extra context explicit.

The example also asks whether an observer can distinguish executions after one candidate session key is substituted for another. That cannot be answered by a single knowledge set: it is a relational property over pairs of trace distributions, beyond a single CTL state formula, and should be represented as a separate theorem family.

11.7 Needham–Schroeder: public-key lookup as behaviour

The seven-message public-key example makes key discovery observable. Alice requests Bob's public key from a server, receives a signed binding, sends an encrypted nonce and identity to Bob, and later answers Bob's nonce. Bob performs the symmetric key lookup for Alice.

Actor separation is particularly valuable here:

- the key server grammar promises signed actor–public-key bindings;
- Alice's grammar accepts Bob's key only where the signature verifies and the named actor is Bob;
- Bob imposes the corresponding constraint for Alice;
- the nonce-response messages reuse previously bound values;
- the intruder grammar may substitute any derivable public key or replay any signed binding, but cannot manufacture a server signature.

```
receive (sign serverSigningKey (tuple [actor bob, publicKey "pkb"]))
  where_ verifiesWith serverPublicKey
  where_ field "actor" == bob

send (encrypt pkb (tuple [nonce na, actor alice]))

receive (encrypt aliceSecretView (tuple [nonce na, nonce "nb"]))
  where_ field "na" == na
```

This demonstrates why “all behaviour is observable” must not mean “all plaintext is public.” The globally recorded event exposes endpoints, timing, kind, and wire bytes. Which symbolic values an actor can recover from those bytes is governed by the deduction relation.

11.8 What these exercises demand from the library

The three case studies produce a concrete implementation checklist:

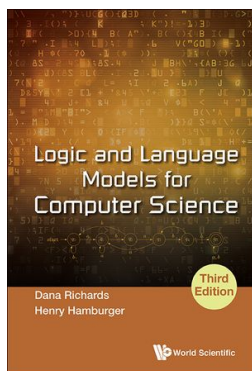
1. actor-local grammar definitions and trace projection;
2. synchronization by common event identity and bytes;
3. typed symbolic variables, exports, imports, and prefix-dependent guards;
4. a transport-neutral term-to-byte relation;
5. explicit initial knowledge and compromise assumptions;
6. attacker closure under symbolic construction and decryption;
7. per-value knowledge sets, agreement, and relational indistinguishability theorem schemas;

8. counterexamples rendered simultaneously as a global interaction diagram and as each actor's local projection.

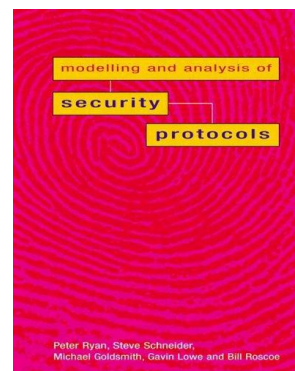
These are not security-specific embellishments. They are a rigorous test of whether multiparty context-sensitive grammars can remain modular while expressing the information flow on which real behavioural claims depend.

Referenced Books

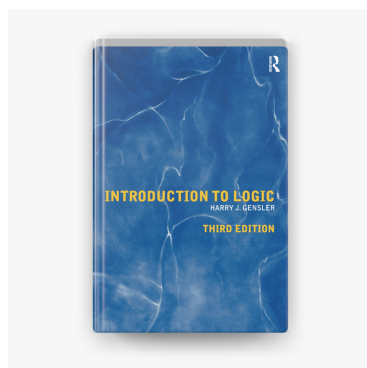
These cover snapshots identify the editions of the books referenced in this volume. They are reproduced at small size for bibliographic identification.



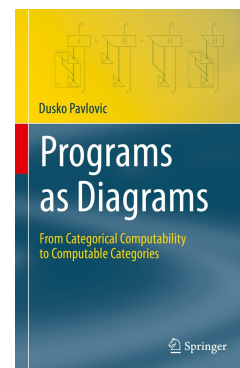
Logic and Language Models for Computer Science
Dana Richards and Henry Hamburger [9]



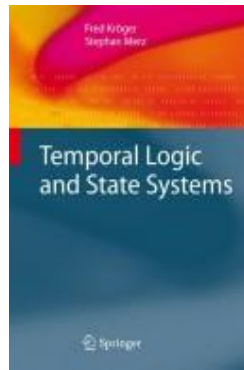
The Modelling and Analysis of Security Protocols
Peter Ryan, Steve Schneider, Michael Goldsmith,
Gavin Lowe, and Bill Roscoe [10]



Introduction to Logic
Harry J. Gensler [3]



Programs as Diagrams
Dusko Pavlovic [8]



Temporal Logic and State Systems
Fred Kröger and Stephan Merz [5]

Cover-image sources, accessed September 25, 2026: World Scientific edition via Kobo for Richards and Hamburger; Open Library for Ryan et al.; Apple Books for Gensler; and Kobo's Springer edition for Pavlovic. The Kröger and Merz cover snapshot is from Open Library. Cover artwork remains the property of its respective publisher.

Bibliography

- [1] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.
- [2] Robert Fielding and LeanFM contributors. LeanFM: Grammar-based formal methods in Lean. GitHub repository, 2026. URL <https://github.com/rfielding/leanFM>. Accessed 2026-09-21.
- [3] Harry J. Gensler. *Introduction to Logic*. Routledge, 3 edition, 2017. ISBN 978-1-138-91058-4. doi: 10.4324/9781315693361.
- [4] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, Berlin, Heidelberg, 2007. ISBN 978-3-540-31010-5. doi: 10.1007/978-3-540-31010-5.
- [5] Fred Kr"oger and Stephan Merz. *Temporal Logic and State Systems*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2008. ISBN 978-3-540-68635-4. doi: 10.1007/978-3-540-68635-4.
- [6] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN 978-0-321-14306-8.
- [7] Daniel A. Menascé and Virgilio A. F. Almeida. *Capacity Planning for Web Services: Metrics, Models, and Methods*. Prentice Hall PTR, Upper Saddle River, NJ, 2002. ISBN 978-0-13-065903-3.
- [8] Dusko Pavlovic. *Programs as Diagrams: From Categorical Computability to Computable Categories*. Springer Nature Switzerland, Cham, Switzerland, 2023. ISBN 978-3-031-34826-6. doi: 10.1007/978-3-031-34827-3.
- [9] Dana Richards and Henry Hamburger. *Logic and Language Models for Computer Science*. World Scientific, 3 edition, 2017. ISBN 978-981-322-920-4. doi: 10.1142/10507.
- [10] Peter Y. A. Ryan, Steve A. Schneider, Michael H. Goldsmith, Gavin Lowe, and A. W. Roscoe. *The Modelling and Analysis of Security Protocols: The CSP Approach*. Addison-Wesley, Harlow, England, 2001. ISBN 978-0-201-67471-2.

Index

- agreement, 67
- alphabet
 - vague, 23
- bytes, 64
- chart specification, 32
- composition
 - requirement intersection, 31
- CSP, 63
- CTL, 54
- event, 5
- formal methods, 2
- grammar
 - context-free, 17
 - multiparty, 17
- intruder model, 66
- knowledge
 - derived from events, 49
- Lean, 59
- LTL, 54
- modal logic, 54
- Needham–Schroeder protocol, 68
- OAuth, 40
- observable event, 22
- Otway–Rees protocol, 67
- parameter flow, 19
- past-time logic, 58
- probability, 25
- profit
 - derived from events, 10
- protocol role, 63
- requirements
 - durable, 35
- scenario, 22
- secrecy, 67
- temporal logic, 54
- time, 9
- trace, 5
- vibecoding, 35
- waste
 - derived from events, 10
- XY plot, 10
- Yahalom protocol, 64